

Modelo 4+1 de Vistas Arquitectónicas

Sistema CRTLPyme - Plataforma POS-SaaS

Instituto Profesional DUOC UC **Ingeniería en Informática** **Proyecto Capstone 2025**
Autores: [Nombres del Equipo] **Asignatura:** Arquitectura de Software **Profesor:** [Nombre del Profesor]
Fecha: Octubre 2025

Tabla de Contenidos

- Resumen Ejecutivo
- Introducción
 - 2.1 Contexto del Proyecto
 - 2.2 Objetivos del Documento
 - 2.3 El Modelo 4+1 de Philippe Kruchten
- Vista Lógica
 - 3.1 Descripción General
 - 3.2 Diagrama de Componentes
 - 3.3 Diagrama de Clases del Dominio
 - 3.4 Patrones de Diseño Aplicados
- Vista de Procesos
 - 4.1 Descripción General
 - 4.2 Proceso: Registro de Venta (POS)
 - 4.3 Proceso: Onboarding de Cliente
 - 4.4 Proceso: Facturación de Suscripciones
 - 4.5 Concurrencia y Sincronización
- Vista de Desarrollo
 - 5.1 Descripción General
 - 5.2 Estructura de Paquetes
 - 5.3 Organización del Código
 - 5.4 Gestión de Dependencias
- Vista Física
 - 6.1 Descripción General
 - 6.2 Diagrama de Despliegue en GCP
 - 6.3 Componentes de Infraestructura
 - 6.4 Estrategia de Escalabilidad
- Vista de Escenarios (Casos de Uso)
 - 7.1 Descripción General
 - 7.2 Actores del Sistema
 - 7.3 Diagrama de Casos de Uso
 - 7.4 Especificación de Casos de Uso Principales

- 8. [Decisiones Arquitectónicas](#)
 - 8.1 [Arquitectura Serverless](#)
 - 8.2 [Multi-Tenancy](#)
 - 8.3 [Seguridad](#)
 - 8.4 [Escalabilidad y Rendimiento](#)
 - 9. [Conclusiones](#)
 - 10. [Referencias Bibliográficas](#)
 - 11. [Anexos](#)
-

Resumen Ejecutivo

El presente documento describe la arquitectura del sistema **CRTLPyMe**, una plataforma SaaS (Software as a Service) de punto de venta e inventario orientada a pequeños comercios chilenos como tiendas de abarrotes, kioscos y minimarkets.

La arquitectura se documenta siguiendo el **Modelo 4+1 de Vistas Arquitectónicas** propuesto por Philippe Kruchten (1995), que permite representar el sistema desde múltiples perspectivas complementarias: lógica, procesos, desarrollo, física y escenarios.

Características principales del sistema:

- Arquitectura cloud-native serverless sobre Google Cloud Platform (GCP)
 - Multi-tenancy con aislamiento de datos por organización
 - Stack tecnológico moderno: Next.js 15, React 19, TypeScript, PostgreSQL
 - Gestión de ventas, inventario, clientes y suscripciones
 - Cálculo automático del punto de equilibrio financiero
 - Sistema de roles y permisos (ADMIN, CASHIER, MANAGER)
-

Introducción

Contexto del Proyecto

CRTLPyMe surge como respuesta a la necesidad de digitalización de pequeños comercios en Chile, que actualmente operan con sistemas manuales o semi-manuales para la gestión de ventas e inventario. Estos negocios enfrentan problemas recurrentes:

- **Falta de control:** Desconocimiento del inventario real, diferencias de caja frecuentes
- **Gestión ineficiente:** Decisiones basadas en intuición sin datos históricos
- **Viabilidad financiera:** Desconocimiento del punto de equilibrio y márgenes reales

El proyecto se desarrolla como parte del programa **Capstone 2025** de la carrera de Ingeniería en Informática del Instituto Profesional DUOC UC, demostrando competencias en:

- Diseño de arquitecturas de software escalables
- Desarrollo de aplicaciones cloud-native
- Implementación de sistemas multi-tenant
- Integración de servicios de pago (Transbank)
- Aplicación de mejores prácticas de ingeniería de software

Objetivos del Documento

Este documento tiene como objetivos:

1. **Documentar la arquitectura completa** del sistema CRTLPyme utilizando el estándar académico del Modelo 4+1
2. **Justificar las decisiones arquitectónicas** tomadas durante el diseño del sistema
3. **Facilitar la comunicación** entre los stakeholders del proyecto (equipo de desarrollo, profesores, evaluadores)
4. **Servir como guía de implementación** para el equipo de desarrollo
5. **Demostrar competencias** en diseño arquitectónico de sistemas de software complejos

El Modelo 4+1 de Philippe Kruchten

El **Modelo 4+1** es un marco de referencia para describir la arquitectura de sistemas de software intensivos, propuesto por Philippe Kruchten en 1995. Este modelo organiza la descripción arquitectónica en cinco vistas complementarias:

Las Cuatro Vistas Principales:

1. **Vista Lógica (Logical View)**
 - **Audiencia:** Usuarios finales, analistas
 - **Foco:** Funcionalidad del sistema
 - **Elementos:** Clases, objetos, componentes, paquetes
 - **Pregunta clave:** “¿Qué hace el sistema?”
2. **Vista de Procesos (Process View)**
 - **Audiencia:** Integradores, desarrolladores de rendimiento
 - **Foco:** Comportamiento dinámico, concurrencia
 - **Elementos:** Procesos, threads, flujos de comunicación
 - **Pregunta clave:** “¿Cómo funciona el sistema en tiempo de ejecución?”
3. **Vista de Desarrollo (Development View)**
 - **Audiencia:** Programadores, gestores de software
 - **Foco:** Organización del código
 - **Elementos:** Módulos, paquetes, bibliotecas, capas
 - **Pregunta clave:** “¿Cómo está organizado el código fuente?”
4. **Vista Física (Physical View)**
 - **Audiencia:** Ingenieros de sistemas, administradores
 - **Foco:** Topología del sistema, despliegue
 - **Elementos:** Nodos físicos, redes, servicios
 - **Pregunta clave:** “¿Dónde se ejecuta el sistema?”

La Vista Adicional:

1. **Vista de Escenarios (Scenarios/Use Cases)**
 - **Audiencia:** Todos los stakeholders
 - **Foco:** Casos de uso, requisitos
 - **Elementos:** Actores, casos de uso, flujos
 - **Pregunta clave:** “¿Por qué está diseñado así?”
 - **Propósito:** Unificar y validar las otras cuatro vistas

El símbolo “+1” representa esta quinta vista que sirve como elemento integrador, permitiendo identificar elementos arquitectónicos a partir de escenarios concretos de uso.

Ventajas del Modelo 4+1:

- Separación de concerns por audiencia
 - Validación cruzada entre vistas
 - Comunicación efectiva con diferentes stakeholders
 - Soporte para sistemas complejos y distribuidos
-

Vista Lógica

3.1 Descripción General

La Vista Lógica describe la **estructura funcional del sistema**, identificando los componentes principales, sus responsabilidades y las relaciones entre ellos. Esta vista está orientada a los usuarios finales y analistas de negocio, respondiendo a la pregunta: “¿Qué hace el sistema?”

CRTLPy se estructura siguiendo una **arquitectura en capas y orientada a servicios**, con separación clara de responsabilidades:

Capas Arquitectónicas:**1. Capa de Presentación (Presentation Layer)**

- Interfaz de usuario construida con Next.js y React
- Componentes reutilizables con Tailwind CSS
- Client-side state management
- Responsive design para múltiples dispositivos

2. Capa de Aplicación (Application Layer)

- Next.js API Routes (serverless functions)
- Lógica de negocio y orquestación
- Validación de datos
- Manejo de autenticación y autorización

3. Capa de Dominio (Domain Layer)

- Entidades del negocio (Tenant, User, Product, Sale)
- Reglas de negocio core
- Servicios de dominio (cálculo de punto de equilibrio)

4. Capa de Persistencia (Persistence Layer)

- Prisma ORM como abstracción de base de datos
- Repositorios para acceso a datos
- Gestión de transacciones

5. Capa de Infraestructura (Infrastructure Layer)

- Integraciones externas (Transbank, Email)
- Servicios de autenticación (NextAuth.js)
- Logging y monitoreo

3.2 Diagrama de Componentes

```

graph TB
  subgraph "Presentation Layer"
    UI[UI Components<br/>Next.js/React]
    Pages[Pages & Routes]
  end

  subgraph "Application Layer"
    API[API Routes<br/>Serverless]
    Auth[Authentication<br/>NextAuth.js]
    Middleware[Middleware<br/>Authorization]
  end

  subgraph "Domain Layer"
    TenantSvc[Tenant Service]
    ProductSvc[Product Service]
    SaleSvc[Sales Service]
    SubsSvc[Subscription Service]
    BreakevenSvc[Breakeven Service]
    FreqCustomerSvc[Frequent Customer Service<br/>OPTIONAL]
  end

  subgraph "Persistence Layer"
    Prisma[Prisma ORM]
    Repos[Repositories]
  end

  subgraph "Infrastructure Layer"
    Transbank[Transbank Integration]
    Email[Email Service]
    Storage[File Storage]
  end

  subgraph "External Systems"
    DB[(PostgreSQL<br/>Cloud SQL)]
    CDN[Cloud CDN]
    Monitoring[Cloud Monitoring]
  end

  UI --> Pages
  Pages --> API
  API --> Auth
  API --> Middleware
  Middleware --> TenantSvc
  Middleware --> ProductSvc
  Middleware --> SaleSvc
  Middleware --> SubsSvc

  TenantSvc --> BreakevenSvc
  TenantSvc --> FreqCustomerSvc
  SaleSvc --> ProductSvc
  SaleSvc --> FreqCustomerSvc
  SubsSvc --> Transbank
  FreqCustomerSvc --> Repos

  TenantSvc --> Repos
  ProductSvc --> Repos
  SaleSvc --> Repos
  SubsSvc --> Repos

  Repos --> Prisma
  Prisma --> DB

```

```
Email -.-> TenantSvc
Storage -.-> UI
CDN -.-> UI
Monitoring -.-> API

style UI fill:#e1f5ff
style API fill:#fff4e1
style TenantSvc fill:#e8f5e9
style Prisma fill:#f3e5f5
style DB fill:#ffebee
```

Descripción de Componentes Principales:

1. UI Components (Capa de Presentación)

- **Responsabilidad:** Renderización de interfaz de usuario, manejo de eventos, validación client-side
- **Tecnologías:** React 19, TypeScript, Tailwind CSS, shadcn/ui
- **Patrones:** Component composition, hooks, controlled components

2. API Routes (Capa de Aplicación)

- **Responsabilidad:** Endpoints serverless, orquestación de servicios, validación de requests
- **Tecnologías:** Next.js App Router, Zod validation
- **Patrones:** RESTful API, Request/Response pattern

3. Services (Capa de Dominio)

- **Tenant Service:** Gestión multi-tenant, configuración de organizaciones
- **Product Service:** CRUD de productos, gestión de inventario, alertas de stock
- **Sales Service:** Registro de ventas, cálculo de totales, actualización de inventario
- **Subscription Service:** Manejo de suscripciones, integración con Transbank
- **Breakeven Service:** Cálculo del punto de equilibrio, análisis financiero
- **Frequent Customer Service (OPCIONAL):** Gestión de clientes frecuentes, cálculo de tramos de descuento, aplicación automática de descuentos, reseteo mensual de acumulaciones

4. Prisma ORM (Capa de Persistencia)

- **Responsabilidad:** Abstracción de base de datos, migraciones, type-safety
- **Tecnologías:** Prisma Client, Prisma Migrate
- **Patrones:** Repository, Unit of Work, Data Mapper

5. Infrastructure Services

- **Transbank Integration:** Pagos recurrentes, inscripción de tarjetas (sandbox)
- **Email Service:** Notificaciones transaccionales
- **File Storage:** Almacenamiento de archivos adjuntos

3.3 Diagrama de Clases del Dominio


```
classDiagram
class Tenant {
+String id
+String name
+String businessType
+String taxId
+DateTime createdAt
+Boolean active
+getUsers()
+getProducts()
+getSales()
+calculateBreakeven()
}

class User {
+String id
+String email
+String name
+UserRole role
+String tenantId
+DateTime createdAt
+Boolean active
+can(permission)
+getTenant()
}

class Product {
+String id
+String sku
+String barcode
+String name
+Decimal cost
+Decimal price
+Int stock
+Int minStock
+String tenantId
+calculateMargin()
+isLowStock()
+updateStock(quantity)
}

class Sale {
+String id
+String tenantId
+String userId
+DateTime date
+Decimal subtotal
+Decimal tax
+Decimal total
+PaymentMethod paymentMethod
+addItem(product, quantity)
+calculateTotal()
+complete()
}

class SaleItem {
+String id
+String saleId
+String productId
+Int quantity
+Decimal unitPrice
+Decimal unitCost
```

```

        +Decimal subtotal
        +calculateSubtotal()
    }

    class Customer {
        +String id
        +String name
        +String phone
        +String email
        +String tenantId
        +getSubscriptions()
        +getTotalDebt()
    }

    class Subscription {
        +String id
        +String customerId
        +String tenantId
        +SubscriptionType type
        +Decimal amount
        +Date startDate
        +Date nextPaymentDate
        +SubscriptionStatus status
        +processPayment()
        +calculateNextPayment()
    }

    class SubscriptionPayment {
        +String id
        +String subscriptionId
        +Decimal amount
        +DateTime paymentDate
        +PaymentStatus status
        +String transactionId
    }

    class FixedExpense {
        +String id
        +String tenantId
        +String name
        +Decimal amount
        +ExpenseFrequency frequency
        +DateTime startDate
        +DateTime endDate
        +Boolean isActive
        +normalizeToMonthly()
    }

    class VariableExpense {
        +String id
        +String tenantId
        +String concept
        +Decimal amount
        +DateTime date
        +ExpenseCategory category
        +String productId
        +String saleId
        +String userId
        +getMonthlyTotal()
    }

    class BreakevenCalculation {
        +String id

```

```

+String tenantId
+String period
+DateTime calculationDate
+Decimal totalFixedCosts
+Decimal totalVariableCosts
+Decimal totalSales
+Decimal grossMargin
+Decimal breakevenPoint
+Int breakevenDays
+Decimal currentProgress
+Boolean isAchieved
+calculateBreakeven()
+willAchieveBreakeven()
+generateRecommendations()
}

Tenant "1" --> "*" User : has
Tenant "1" --> "*" Product : manages
Tenant "1" --> "*" Sale : records
Tenant "1" --> "*" Customer : serves
Tenant "1" --> "*" FixedExpense : has
Tenant "1" --> "*" VariableExpense : has
Tenant "1" --> "*" BreakevenCalculation : calculates

User "1" --> "*" Sale : creates
User "1" --> "*" VariableExpense : registers

Sale "1" --> "*" SaleItem : contains
Sale "1" --> "*" VariableExpense : may have
SaleItem "*" --> "1" Product : references

Product "1" --> "*" VariableExpense : may have

Customer "1" --> "*" Subscription : has
Subscription "1" --> "*" SubscriptionPayment : generates

<<enumeration>> UserRole {
    ADMIN
    CASHIER
    MANAGER
    INVENTARIO
}

<<enumeration>> PaymentMethod {
    CASH
    DEBIT_CARD
    CREDIT_CARD
}

<<enumeration>> SubscriptionType {
    DAILY
    WEEKLY
    MONTHLY
}

<<enumeration>> ExpenseFrequency {
    DAILY
    WEEKLY
    MONTHLY
    YEARLY
}

<<enumeration>> ExpenseCategory {

```

```

    PRODUCT_COST
    COMMISSION
    TRANSPORT
    PACKAGING
    OTHER
}

```

Descripción de Entidades Principales:

1. Tenant (Multi-tenancy)

- **Propósito:** Representa una organización/empresa cliente del sistema
- **Responsabilidades:**
 - Aislamiento de datos por cliente
 - Configuración de organización
 - Gestión de suscripción al servicio SaaS
- **Relaciones:** Padre de todas las entidades del dominio

2. User (Gestión de Usuarios)

- **Propósito:** Usuario del sistema con roles específicos
- **Responsabilidades:**
 - Autenticación y autorización
 - Control de acceso basado en roles (RBAC)
 - Trazabilidad de operaciones
- **Roles:**
 - **ADMIN** : Administrador del negocio, acceso completo
 - **CASHIER** : Cajero, solo puede registrar ventas
 - **MANAGER** : Supervisor, puede ver reportes
 - **INVENTARIO** : Encargado de inventario y gastos

3. Product (Gestión de Inventario)

- **Propósito:** Producto vendible del negocio
- **Responsabilidades:**
 - Gestión de stock en tiempo real
 - Cálculo de márgenes de ganancia
 - Alertas de stock bajo (minStock)
 - Trazabilidad de costos vs. precios

4. Sale y SaleItem (Punto de Venta)

- **Propósito:** Registro de transacciones de venta
- **Responsabilidades:**
 - Registro atómico de ventas
 - Actualización automática de inventario
 - Cálculo de totales con impuestos
 - Soporte para múltiples métodos de pago
- **Patrón:** Aggregate root (Sale) con entidades asociadas (SaleItem)

5. Customer y Subscription (Gestión de Clientes)

- **Propósito:** Clientes y sus suscripciones de productos
- **Responsabilidades:**
 - Registro de clientes frecuentes
 - Gestión de suscripciones recurrentes
 - Control de pagos y deudas
 - Renovación automática

6. FixedExpense (Gastos Fijos)

- **Propósito:** Registro de gastos operacionales recurrentes del negocio
- **Responsabilidades:**
 - Almacenar gastos fijos (arriendo, servicios, sueldos)
 - Normalizar gastos a frecuencia mensual
 - Servir como base para cálculo del punto de equilibrio
 - Permitir activación/desactivación temporal

7. VariableExpense (Gastos Variables)

- **Propósito:** Registro de gastos que varían con el volumen de ventas
- **Responsabilidades:**
 - Registrar gastos variables (comisiones, transporte, embalaje)
 - Asociar gastos a productos o ventas específicas
 - Calcular costos variables totales del período
 - Servir como base para cálculo del margen de contribución

8. BreakevenCalculation (Cálculo de Punto de Equilibrio)

- **Propósito:** Almacenar cálculos históricos del punto de equilibrio
- **Responsabilidades:**
 - Calcular punto de equilibrio considerando todos los costos
 - Determinar progreso hacia el equilibrio
 - Proyectar si se alcanzará el equilibrio en el mes
 - Generar recomendaciones para mejorar rentabilidad
 - Mantener histórico mensual para análisis de tendencias

3.4 Patrones de Diseño Aplicados

Patrones Estructurales:

1. Repository Pattern

- **Problema:** Acoplamiento directo entre lógica de negocio y acceso a datos
- **Solución:** Abstracción del acceso a datos mediante interfaces
- **Implementación:**

```
interface IProductRepository {
  findById(id: string): Promise<Product | null>;
  findByTenant(tenantId: string): Promise<Product[]>;
  create(product: Product): Promise<Product>;
  update(id: string, data: Partial<Product>): Promise<Product>;
  delete(id: string): Promise<void>;
}
```

2. Unit of Work Pattern

- **Problema:** Gestión compleja de transacciones que afectan múltiples entidades
- **Solución:** Coordinar cambios y confirmarlos en una sola transacción
- **Implementación:** Prisma transactions

```
await prisma.$transaction(async (tx) => {
  await tx.sale.create({...});
  await tx.product.updateMany({...});
  await tx.saleItem.createMany({...});
});
```

3. Facade Pattern

- **Problema:** Complejidad de múltiples servicios para operaciones comunes
- **Solución:** Interfaz simplificada que orquesta múltiples servicios
- **Implementación:** Service layer que coordina repositories

Patrones de Comportamiento:

4. Strategy Pattern

- **Problema:** Diferentes estrategias de cálculo de precios y descuentos
- **Solución:** Encapsular algoritmos intercambiables
- **Aplicación:** Cálculo de punto de equilibrio, estrategias de descuento

5. Observer Pattern

- **Problema:** Notificaciones cuando cambia el estado del inventario
- **Solución:** Sistema de eventos para alertas de stock bajo
- **Implementación:** React hooks + server events

6. Command Pattern

- **Problema:** Necesidad de deshacer operaciones o implementar transacciones compensatorias
- **Solución:** Encapsular operaciones como objetos
- **Aplicación:** Ajustes de inventario, cancelación de ventas

Patrones Arquitectónicos:

7. Layered Architecture

- **Aplicación:** Separación clara en capas (Presentation, Application, Domain, Persistence, Infrastructure)
- **Beneficio:** Bajo acoplamiento, alta cohesión, facilidad de testing

8. Domain-Driven Design (DDD) Lite

- **Aplicación:** Modelado centrado en el dominio del negocio
- **Elementos:**
 - Entities: Tenant, User, Product, Sale
 - Value Objects: Money, SKU, Barcode
 - Aggregates: Sale + SaleItems
 - Domain Services: BreakevenCalculator, InventoryAlertService

9. Multi-Tenancy Pattern

- **Tipo:** Shared Database, Separate Schema (tenant_id discriminator)
- **Implementación:** Middleware que inyecta tenantId en todas las queries
- **Seguridad:** Row-Level Security a nivel de aplicación

Vista de Procesos

4.1 Descripción General

La Vista de Procesos describe el **comportamiento dinámico del sistema** en tiempo de ejecución, incluyendo:

- Flujos de control entre componentes
- Concurrencia y paralelismo
- Sincronización y comunicación entre procesos
- Manejo de transacciones

Esta vista está orientada a integradores y desarrolladores, respondiendo: “**¿Cómo funciona el sistema en ejecución?**”

Características del modelo de procesos de CRTLPyme:

1. **Arquitectura Serverless:** Funciones stateless invocadas por eventos HTTP
2. **Transacciones ACID:** Garantías de consistencia en operaciones críticas
3. **Operaciones Asíncronas:** Jobs programados para facturación y alertas
4. **Multi-tenancy Concurrente:** Múltiples tenants operando simultáneamente
5. **Idempotencia:** Protección contra operaciones duplicadas

4.2 Proceso: Registro de Venta (POS)

Este es el proceso core del sistema, donde un cajero registra una venta en el punto de venta.

Diagrama de Secuencia:


```

sequenceDiagram
    actor Cashier as Cajero
    participant UI as POS UI
    participant API as Sales API
    participant Auth as Auth Service
    participant SalesSvc as Sales Service
    participant ProductSvc as Product Service
    participant DB as PostgreSQL

    Cashier->>UI: Escanea código de barras
    UI->>API: GET /api/products/by-barcode/{code}
    API->>Auth: Verificar token JWT
    Auth-->>API: Token válido + tenantId
    API->>ProductSvc: findByBarcode(code, tenantId)
    ProductSvc->>DB: SELECT WHERE barcode AND tenant_id
    DB-->>ProductSvc: Product data
    ProductSvc-->>API: Product
    API-->>UI: Product details

    UI->>UI: Agregar a carrito
    Note over UI: Repetir para cada producto

    Cashier->>UI: Confirmar venta
    UI->>API: POST /api/sales {items, paymentMethod}
    API->>Auth: Verificar token + permisos
    Auth-->>API: Autorizado

    API->>SalesSvc: createSale(data)
    SalesSvc->>DB: BEGIN TRANSACTION

    SalesSvc->>DB: INSERT INTO sales
    DB-->>SalesSvc: sale_id

    loop Para cada item
        SalesSvc->>DB: INSERT INTO sale_items
        SalesSvc->>ProductSvc: decrementStock(productId, qty)
        ProductSvc->>DB: UPDATE products SET stock = stock - qty<br/>WHERE id = ? FOR
    UPDATE

        alt Stock suficiente
            DB-->>ProductSvc: OK
        else Stock insuficiente
            ProductSvc-->>SalesSvc: InsufficientStockError
            SalesSvc->>DB: ROLLBACK
            SalesSvc-->>API: Error 400
            API-->>UI: "Stock insuficiente"
        end
    end

    SalesSvc->>DB: COMMIT
    DB-->>SalesSvc: Success

    SalesSvc->>ProductSvc: checkLowStock(tenantId)
    ProductSvc->>DB: SELECT WHERE stock < minStock
    alt Hay productos con stock bajo
        ProductSvc-->>SalesSvc: Low stock alerts
        SalesSvc->>API: Trigger notification
    end

    SalesSvc-->>API: Sale created
    API-->>UI: Sale ID + receipt data
    UI-->>Cashier: Mostrar comprobante

```

Aspectos Destacados:

1. Transaccionalidad:

- Uso de transacciones ACID para garantizar consistencia
- Rollback automático si falla actualización de inventario
- Bloqueo pesimista (FOR UPDATE) para evitar race conditions

2. Validaciones:

- Autenticación JWT en cada request
- Verificación de permisos según rol
- Validación de stock antes de confirmar venta

3. Eventos y Notificaciones:

- Detección automática de stock bajo después de cada venta
- Trigger de alertas asíncronas

4. Manejo de Errores:

- Validación temprana de stock
- Rollback transaccional en caso de error
- Mensajes descriptivos al usuario

4.3 Proceso: Onboarding de Cliente

Proceso de registro de un nuevo cliente (tenant) en la plataforma SaaS.

Diagrama de Flujo:

```

stateDiagram-v2
    [*] --> SelectPlan: Usuario visita landing

    SelectPlan --> RegisterBusiness: Selecciona plan
    note right of SelectPlan
        2 cajas incluidas por defecto
        Opción de cajas adicionales
    end note

    RegisterBusiness --> ConfigureBreakeven: Completa datos del negocio
    note right of RegisterBusiness
        Nombre, RUT, dirección
        Tipo de negocio
        Email administrador
    end note

    ConfigureBreakeven --> SetupPayment: Ingresa costos fijos y margen
    note right of ConfigureBreakeven
        Gastos mensuales fijos
        Margen promedio de productos
        Sistema calcula punto de equilibrio
    end note

    SetupPayment --> CreateTenant: Inscribe tarjeta (Transbank)
    note right of SetupPayment
        Flujo Oneclick Transbank
        Sandbox environment
        Sin cobro inicial
    end note

    CreateTenant --> SendCredentials: Crear tenant y admin user
    note right of CreateTenant
        Transacción atómica:
        - INSERT tenant
        - INSERT user (ADMIN)
        - INSERT breakeven_config
        - INSERT subscription
    end note

    SendCredentials --> [*]: Email con acceso enviado

    CreateTenant --> RollbackTenant: Error en creación
    RollbackTenant --> [*]: Notificar error
  
```

Diagrama de Secuencia Detallado:

```

sequenceDiagram
    actor Client as Nuevo Cliente
    participant Landing as Landing Page
    participant API as Onboarding API
    participant TenantSvc as Tenant Service
    participant Transbank as Transbank API
    participant Email as Email Service
    participant DB as PostgreSQL

    Client->>Landing: Accede a registro
    Landing->>API: GET /api/onboarding/plans
    API-->>Landing: Lista de planes disponibles

    Client->>Landing: Selecciona plan y llena formulario
    Landing->>API: POST /api/onboarding/register<br/>{businessData, adminData, plan}

    API->>API: Validar datos (Zod schema)

    API->>DB: BEGIN TRANSACTION

    API->>TenantSvc: createTenant(businessData)
    TenantSvc->>DB: INSERT INTO tenants
    DB-->>TenantSvc: tenant_id

    TenantSvc->>DB: INSERT INTO users (role=ADMIN)
    DB-->>TenantSvc: user_id

    TenantSvc->>DB: INSERT INTO breakeven_configs
    TenantSvc->>DB: INSERT INTO subscriptions<br/>(status=PENDING_PAYMENT)

    API->>Transbank: POST /oneclick/inscriptions<br/>{username, email, returnUrl}
    Transbank-->>API: token + inscriptionUrl

    API->>DB: UPDATE subscriptions<br/>SET transbank_token = token

    API->>DB: COMMIT
    DB-->>API: Success

    API-->>Landing: {tenantId, inscriptionUrl}
    Landing-->>Client: Redirigir a Transbank

    Client->>Transbank: Completa inscripción de tarjeta
    Transbank->>API: GET /api/onboarding/oneclick/callback<br/>?token={token}&TBK_TOKEN={tbk_token}

    API->>Transbank: POST /oneclick/inscriptions/confirm<br/>{token, tbk_token}
    Transbank-->>API: {responseCode, authCode, cardType}

    alt Inscripción exitosa
        API->>DB: UPDATE subscriptions<br/>SET status = ACTIVE,<br/>payment_token = tbk_token
        API->>Email: sendWelcomeEmail(admin_email, credentials)
        Email-->>API: Email sent
        API-->>Landing: Success
        Landing-->>Client: "¡Bienvenido! Revisa tu email"
    else Inscripción fallida
        API->>DB: UPDATE subscriptions<br/>SET status = PAYMENT_FAILED
        API-->>Landing: Error
        Landing-->>Client: "Error en pago, intenta nuevamente"
    end
end

```

4.4 Proceso: Facturación de Suscripciones

Proceso automatizado que se ejecuta diariamente para cobrar las suscripciones vencidas.

Diagrama de Actividad:

```

flowchart TD
    Start([Cron Job Diario<br/>00:00 UTC]) --> GetDue[Obtener suscripciones<br/>con ne<br/>xt_billing_date = today]

    GetDue --> CheckCount{¿Hay suscripciones<br/>por cobrar?}

    CheckCount -->|No| End([Fin])
    CheckCount -->|Sí| LoopStart[Para cada suscripción]

    LoopStart --> CheckActive{¿Estado = ACTIVE?}
    CheckActive -->|No| SkipTenant[Saltar tenant]
    CheckActive -->|Sí| CheckToken{¿Tiene token<br/>Transbank?}

    CheckToken -->|No| MarkFailed[Marcar como<br/>PAYMENT_FAILED]
    CheckToken -->|Sí| CallTransbank[Transbank: oneclick/charge]

    CallTransbank --> CheckResponse{¿Cobro exitoso?}

    CheckResponse -->|Sí| RecordPayment[INSERT subscription_payment<br/>status = COM-  
PLETED]
    RecordPayment --> UpdateNext[UPDATE subscription<br/>next_billing_date += inter-  
val]
    UpdateNext --> SendReceipt[Enviar email<br/>con comprobante]

    CheckResponse -->|No| RetryLogic{¿Intentos < 3?}
    RetryLogic -->|Sí| ScheduleRetry[Programar reintento<br/>en 24 horas]
    RetryLogic -->|No| SuspendTenant[UPDATE subscription<br/>status = SUSPENDED]
    SuspendTenant --> NotifyAdmin[Notificar admin<br/>del tenant]

    SendReceipt --> NextSub[Siguiente suscripción]
    SkipTenant --> NextSub
    MarkFailed --> NextSub
    ScheduleRetry --> NextSub
    NotifyAdmin --> NextSub

    NextSub --> CheckMore{¿Más suscripciones?}
    CheckMore -->|Sí| LoopStart
    CheckMore -->|No| GenerateReport[Generar reporte diario]

    GenerateReport --> End

    style Start fill:#e1f5ff
    style End fill:#e1f5ff
    style CallTransbank fill:#fff4e1
    style RecordPayment fill:#e8f5e9
    style SuspendTenant fill:#ffebee
  
```

Consideraciones de Concurrencia:

1. Bloqueos Distribuidos:

```
// Evitar ejecuciones concurrentes del job
const lock = await acquireDistributedLock('billing:daily-run');
if (!lock) {
  console.log('Another instance is running');
  return;
}
```

2. Procesamiento por Lotes:

```
// Procesar en chunks para evitar timeout
const BATCH_SIZE = 50;
const subscriptions = await getSubscriptionsDue(today);

for (let i = 0; i < subscriptions.length; i += BATCH_SIZE) {
  const batch = subscriptions.slice(i, i + BATCH_SIZE);
  await Promise.allSettled(
    batch.map(sub => processBilling(sub))
  );
}
```

3. Idempotencia:

```
// Tabla de idempotencia para evitar cobros duplicados
await prisma.idempotencyKey.create({
  data: {
    key: `billing:${subscriptionId}:${billingDate}`,
    status: 'PROCESSING'
  }
});
```

4.5 Concurrencia y Sincronización

Escenarios de Concurrencia:

1. Ventas Concurrentes del Mismo Producto

Problema: Dos cajeros venden el último ítem en stock simultáneamente

Solución:

```
-- Bloqueo pesimista en PostgreSQL
BEGIN;
SELECT stock FROM products
WHERE id = :productId AND tenant_id = :tenantId
FOR UPDATE; -- Lock exclusivo

UPDATE products
SET stock = stock - :quantity
WHERE id = :productId
AND stock >= :quantity; -- Verificación atómica

COMMIT;
```

2. Actualización Concurrente de Inventario

Problema: Venta simultánea con ajuste manual de inventario

Solución: Isolation level `SERIALIZABLE` o `REPEATABLE READ`

```
await prisma.$transaction(
  async (tx) => {
    const product = await tx.product.findUnique({
      where: { id: productId }
    });

    if (product.stock < quantity) {
      throw new InsufficientStockError();
    }

    await tx.product.update({
      where: { id: productId },
      data: { stock: { decrement: quantity } }
    });
  },
  {
    isolationLevel: 'Serializable'
  }
);
```

3. Procesamiento Paralelo de Reportes

Patrón: Worker pool pattern para generación de reportes pesados

```
graph LR
  API[API Request] --> Queue[Job Queue]
  Queue --> W1[Worker 1]
  Queue --> W2[Worker 2]
  Queue --> W3[Worker 3]
  W1 --> DB[(Database)]
  W2 --> DB
  W3 --> DB
  W1 --> S3[Cloud Storage]
  W2 --> S3
  W3 --> S3

  style Queue fill:#fff4e1
  style S3 fill:#e8f5e9
```

Estrategias de Sincronización:

1. Optimistic Locking (para operaciones frecuentes con baja contención):

```
model Product {
  id      String    @id
  version Int        @default(0) // Version field
  // ... otros campos
}
```



```

await prisma.product.update({
  where: {
    id: productId,
    version: currentVersion // Verificar versión
  },
  data: {
    stock: newStock,
    version: { increment: 1 } // Incrementar versión
  }
});

```

2. Pessimistic Locking (para operaciones críticas):

- Ya mostrado en ejemplos de ventas

3. Distributed Locking (para jobs programados):

```

import { Redis } from '@upstash/redis';

async function acquireLock(key: string, ttl: number = 60) {
  const redis = new Redis({ /* config */ });
  const lockValue = crypto.randomUUID();

  const acquired = await redis.set(
    `lock:${key}`,
    lockValue,
    { nx: true, ex: ttl }
  );

  return acquired ? lockValue : null;
}

```

Vista de Desarrollo

5.1 Descripción General

La Vista de Desarrollo describe la **organización del código fuente**, incluyendo:

- Estructura de directorios y paquetes
- Dependencias entre módulos
- Convenciones de naming y coding
- Gestión de configuración

Esta vista está orientada a programadores y gestores de software, respondiendo: “¿Cómo está organizado el código?”

Principios de Organización:

1. **Separation of Concerns:** Cada módulo tiene una responsabilidad clara
2. **Convention over Configuration:** Estructura estandarizada de Next.js
3. **Modularidad:** Componentes y servicios reutilizables
4. **Type Safety:** TypeScript en todo el stack
5. **Code Ownership:** Directorios mapeados a áreas funcionales

5.2 Estructura de Paquetes

```
graph TB
  subgraph "Root Directory"
    app[app/<br/>Next.js App Router]
    components[components/<br/>React Components]
    lib[lib/<br/>Utilities & Config]
    prisma[prisma/<br/>Database Schema]
    types[types/<br/>TypeScript Types]
    public[public/<br/>Static Assets]
  end

  subgraph "app/ Structure"
    auth[auth/<br/>Authentication]
    dashboard[dashboard/<br/>Main Dashboard]
    inventory[inventory/<br/>Inventory Management]
    sales[sales/<br/>POS]
    customers[customers/<br/>Customer Management]
    api[api/<br/>API Routes]
  end

  subgraph "lib/ Structure"
    auth_lib[auth/<br/>NextAuth Config]
    prisma_lib[prisma/<br/>Prisma Client]
    utils[utils/<br/>Helper Functions]
    validations[validations/<br/>Zod Schemas]
    services[services/<br/>Business Logic]
  end

  subgraph "components/ Structure"
    ui[ui/<br/>shadcn/ui]
    pos_comp[pos/<br/>POS Components]
    inventory_comp[inventory/<br/>Inventory Components]
    shared[shared/<br/>Shared Components]
  end

  app --> auth
  app --> dashboard
  app --> inventory
  app --> sales
  app --> customers
  app --> api

  lib --> auth_lib
  lib --> prisma_lib
  lib --> utils
  lib --> validations
  lib --> services

  components --> ui
  components --> pos_comp
  components --> inventory_comp
  components --> shared

  style app fill:#e1f5ff
  style lib fill:#e8f5e9
  style components fill:#fff4e1
  style prisma fill:#f3e5f5
```

5.3 Organización del Código

Estructura Completa del Proyecto:

```

crtlpyme-mvp/
├── app/
│   ├── (auth)/
│   │   ├── login/
│   │   │   └── page.tsx
│   │   └── register/
│   │       └── page.tsx
│   ├── (dashboard)/
│   │   ├── layout.tsx
│   │   └── page.tsx
│   ├── inventory/
│   │   ├── page.tsx
│   │   ├── new/
│   │   │   └── page.tsx
│   │   └── [id]/
│   │       ├── page.tsx
│   │       ├── edit/
│   │       └── page.tsx
│   ├── sales/
│   │   ├── page.tsx
│   │   ├── history/
│   │   │   └── page.tsx
│   ├── customers/
│   │   ├── page.tsx
│   │   └── [id]/
│   │       └── page.tsx
│   └── settings/
│       ├── page.tsx
│       ├── users/
│       └── breakeven/
├── api/
│   ├── auth/
│   │   ├── [...nextauth]/
│   │   │   └── route.ts
│   ├── products/
│   │   ├── route.ts
│   │   ├── [id]/
│   │   │   ├── route.ts
│   │   │   ├── by-barcode/
│   │   │   │   └── [code]/
│   │   │   │       └── route.ts
│   ├── sales/
│   │   ├── route.ts
│   │   ├── [id]/
│   │   │   └── route.ts
│   ├── customers/
│   │   └── route.ts
│   └── subscriptions/
│       ├── route.ts
│       └── billing/
│           └── route.ts
└──

```

Next.js App Router
 # Grupo de rutas de autenticación
 # Grupo de rutas protegidas
 # Layout con sidebar
 # Dashboard principal
 # Módulo de inventario
 # Lista de productos
 # Crear producto
 # Ver/editar producto
 # Módulo de ventas (POS)
 # Interfaz POS
 # Historial de ventas
 # Módulo de clientes
 # Configuración
 # API Routes (Serverless)
 # NextAuth config
 # GET /api/products, POST /api/products
 # GET/PUT/DELETE /api/products/:id
 # GET /api/products/by-barcode/:code
 # POST /api/sales
 # GET /api/sales/:id

└─ frequent-customer/	# OPTIONAL
└─┬─ config/	
│ └─ route.ts	# GET/POST/PUT config
└─┬─ tiers/	
│ └─ route.ts	# GET/POST/PUT tiers
└─┬─ [customerId]/	
│ └─ enroll/	
│ │ └─ route.ts	# POST enroll customer
│ └─ stats/	
│ │ └─ route.ts	# GET customer stats
│ └─ history/	
│ │ └─ route.ts	# GET purchase history
└─┬─ reports/	
│ └─ route.ts	# GET reports
└─ layout.tsx	# Root layout
└─ globals.css	# Global styles
└─ providers.tsx	# Context providers
└─ components/	# React Components
└─┬─ ui/	# shadcn/ui components
│ └─ button.tsx	
│ └─ input.tsx	
│ └─ card.tsx	
│ └─ table.tsx	
│ └─ ...	
└─┬─ pos/	# POS-specific components
│ └─ ProductScanner.tsx	
│ └─ Cart.tsx	
│ └─ PaymentModal.tsx	
│ └─ Receipt.tsx	
└─┬─ inventory/	# Inventory components
│ └─ ProductList.tsx	
│ └─ ProductForm.tsx	
│ └─ StockAlert.tsx	
│ └─ ProductImport.tsx	
└─┬─ customers/	# Customer components
│ └─ CustomerList.tsx	
│ └─ CustomerForm.tsx	
│ └─ SubscriptionCard.tsx	
└─┬─ shared/	# Shared components
│ └─ Navbar.tsx	
│ └─ Sidebar.tsx	
│ └─ DataTable.tsx	
│ └─ EmptyState.tsx	
│ └─ LoadingSpinner.tsx	
└─ lib/	# Utilities & Configuration
└─┬─ auth/	
│ └─ nextauth.ts	# NextAuth configuration
│ └─ session.ts	# Session helpers
│ └─ permissions.ts	# RBAC utilities
└─┬─ prisma/	
│ └─ client.ts	# Prisma client singleton
└─┬─ services/	# Business logic services
│ └─ tenant.service.ts	
│ └─ product.service.ts	

sale.service.ts	
subscription.service.ts	
breakeven.service.ts	
frequent-customer.service.ts	# OPTIONAL
validations/	# Zod schemas
product.schema.ts	
sale.schema.ts	
user.schema.ts	
utils/	# Helper functions
format.ts	# Formatters (currency, date)
errors.ts	# Error handling
calculations.ts	# Business calculations
constants.ts	# App constants
prisma/	# Database
schema.prisma	# Prisma schema
seed.ts	# Seed script
migrations/	# Database migrations
types/	# TypeScript Types
api.d.ts	# API types
models.d.ts	# Domain models
next-auth.d.ts	# NextAuth extensions
public/	# Static assets
images/	
icons/	
.env	# Environment variables
.env.example	# Environment template
next.config.js	# Next.js configuration
tailwind.config.ts	# Tailwind configuration
tsconfig.json	# TypeScript configuration
package.json	# Dependencies
README.md	# Documentation

Explicación de Convenciones:

1. Route Groups (app/)

- Paréntesis (auth) y (dashboard) para agrupar sin afectar URLs
- Layout específico por grupo

2. API Routes

- RESTful conventions
- route.ts para handlers HTTP
- Nested routes con [id] para parámetros dinámicos

3. Component Organization

- Por feature/módulo (pos/, inventory/, customers/)
- Componentes UI base en ui/
- Shared components en shared/

4. Business Logic (lib/services/)

- Separada de la UI y API routes
- Reusable y testeable
- Inyección de dependencias (Prisma client)

5.4 Gestión de Dependencias

Stack Tecnológico Detallado:

```
{
  "dependencies": {
    // Framework Core
    "next": "^15.0.0",
    "react": "^19.0.0",
    "react-dom": "^19.0.0",

    // Database & ORM
    "@prisma/client": "^5.0.0",
    "prisma": "^5.0.0",

    // Authentication
    "next-auth": "^5.0.0",
    "bcryptjs": "^2.4.3",
    "jsonwebtoken": "^9.0.0",

    // Validation
    "zod": "^3.22.0",

    // UI Components
    "@radix-ui/react-dialog": "^1.0.5",
    "@radix-ui/react-dropdown-menu": "^2.0.6",
    "@radix-ui/react-select": "^2.0.0",
    "tailwindcss": "^3.4.0",
    "class-variance-authority": "^0.7.0",
    "clsx": "^2.0.0",
    "tailwind-merge": "^2.0.0",

    // Forms
    "react-hook-form": "^7.48.0",
    "@hookform/resolvers": "^3.3.0",

    // State Management
    "zustand": "^4.4.0",

    // Utilities
    "date-fns": "^2.30.0",
    "lucide-react": "^0.290.0"
  },
  "devDependencies": {
    // TypeScript
    "typescript": "^5.2.0",
    "@types/node": "^20.0.0",
    "@types/react": "^19.0.0",

    // Linting & Formatting
    "eslint": "^8.53.0",
    "eslint-config-next": "^15.0.0",
    "prettier": "^3.0.0",

    // Testing
    "@testing-library/react": "^14.0.0",
    "@testing-library/jest-dom": "^6.1.0",
    "vitest": "^0.34.0"
  }
}
```

Diagrama de Dependencias:

```
graph TD
  subgraph "Presentation Layer"
    Next[Next.js 15]
    React[React 19]
    Tailwind[Tailwind CSS]
    RadixUI[Radix UI]
  end

  subgraph "Application Layer"
    NextAuth[NextAuth.js]
    Zod[Zod Validation]
    RHF[React Hook Form]
  end

  subgraph "Data Layer"
    Prisma[Prisma ORM]
    PG[PostgreSQL Client]
  end

  subgraph "External Services"
    Transbank[Transbank SDK]
    Email[Email Service]
  end

  Next --> React
  Next --> NextAuth
  Next --> Prisma

  React --> Tailwind
  React --> RadixUI
  React --> RHF

  RHF --> Zod
  NextAuth --> Prisma

  Prisma --> PG

  Next -.-> Transbank
  Next -.-> Email

  style Next fill:#elf5ff
  style Prisma fill:#e8f5e9
  style NextAuth fill:#fff4e1
```

Políticas de Versionado:

1. Semantic Versioning:

- `^5.0.0` : Acepta minor y patch updates (5.x.x)
- `~5.0.0` : Solo patch updates (5.0.x)
- Dependencias críticas (Next.js, React) con caret (^)

2. Lock Files:

- `package-lock.json` versionado para builds reproducibles
- CI/CD usa `npm ci` para instalación determinística

3. Dependencias de Desarrollo:

- Herramientas de testing y linting solo en devDependencies
- No incluidas en build de producción

4. Auditorías de Seguridad:

```
npm audit fix      # Actualizar dependencias vulnerables
npm outdated      # Verificar versiones obsoletas
```

Vista Física

6.1 Descripción General

La Vista Física describe el **despliegue del sistema en infraestructura física y de red**, incluyendo:

- Topología de hardware y servicios cloud
- Distribución de componentes en nodos
- Configuración de red y seguridad
- Estrategias de escalabilidad y alta disponibilidad

Esta vista está orientada a ingenieros de sistemas y administradores, respondiendo: “**¿Dónde se ejecuta el sistema?**”

CRTLPyme adopta una arquitectura **cloud-native** sobre **Google Cloud Platform (GCP)**, aprovechando servicios managed y serverless para:

- Escalabilidad automática
- Alta disponibilidad
- Reducción de costos operativos
- Despliegue rápido y CI/CD

6.2 Diagrama de Despliegue en GCP

```

graph TB
  subgraph "Internet"
    Users[Usuarios<br/>Web/Mobile]
    Admin[Administradores]
  end

  subgraph "Google Cloud Platform"
    subgraph "Edge Services"
      LB[Cloud Load Balancer<br/>HTTPS]
      CDN[Cloud CDN<br/>Static Assets]
      Armor[Cloud Armor<br/>DDoS Protection]
    end

    subgraph "Compute Layer"
      CR1[Cloud Run<br/>Next.js Instance 1<br/>Region: us-east1]
      CR2[Cloud Run<br/>Next.js Instance 2<br/>Region: us-east1]
      CR3[Cloud Run<br/>Next.js Instance 3<br/>Region: us-east1]
    end

    subgraph "Data Layer"
      SQL[(Cloud SQL<br/>PostgreSQL<br/>Multi-AZ)]
      Storage[Cloud Storage<br/>User Uploads]
    end

    subgraph "Authentication"
      IAP[Identity Platform]
      SM[Secret Manager<br/>API Keys]
    end

    subgraph "Monitoring & Operations"
      Logging[Cloud Logging]
      Monitoring[Cloud Monitoring]
      Trace[Cloud Trace]
    end

    subgraph "CI/CD"
      GH[GitHub Actions]
      Build[Cloud Build]
      Registry[Artifact Registry<br/>Docker Images]
    end
  end

  subgraph "External Services"
    Transbank[Transbank API<br/>Payment Gateway]
    SMTP[Email Service<br/>SendGrid/Gmail]
  end

  Users --> LB
  Admin --> LB

  LB --> Armor
  Armor --> CDN
  CDN --> CR1
  CDN --> CR2
  CDN --> CR3

  CR1 --> SQL
  CR2 --> SQL
  CR3 --> SQL

  CR1 --> Storage
  CR2 --> Storage

```

```

CR3 --> Storage

CR1 --> IAP
CR2 --> IAP
CR3 --> IAP

CR1 --> SM
CR2 --> SM
CR3 --> SM

CR1 --> Transbank
CR2 --> Transbank
CR3 --> Transbank

CR1 --> SMTP
CR2 --> SMTP
CR3 --> SMTP

CR1 --> Logging
CR2 --> Logging
CR3 --> Logging

Logging --> Monitoring
Monitoring --> Trace

GH --> Build
Build --> Registry
Registry --> CR1
Registry --> CR2
Registry --> CR3

style Users fill:#e1f5ff
style LB fill:#fff4e1
style CR1 fill:#e8f5e9
style CR2 fill:#e8f5e9
style CR3 fill:#e8f5e9
style SQL fill:#ffebee
style Storage fill:#f3e5f5
style Transbank fill:#fce4ec

```

6.3 Componentes de Infraestructura

1. Cloud Run (Compute)

Descripción:

Servicio serverless de contenedores con escalado automático.

Especificaciones:

```

service: crtlpyme-app
region: us-east1
container:
  image: gcr.io/crtlpyme/app:latest
  port: 3000

resources:
  cpu: 1
  memory: 512Mi
  maxInstances: 100
  minInstances: 0 # Escala a 0 cuando no hay tráfico

concurrency: 80 # Requests concurrentes por instancia

env:
- name: DATABASE_URL
  valueFrom:
    secretKeyRef: database-url
- name: NEXTAUTH_SECRET
  valueFrom:
    secretKeyRef: nextauth-secret

healthCheck:
  path: /api/health
  initialDelaySeconds: 0
  periodSeconds: 10

```

Ventajas:

- **Escalado automático:** 0 a N instancias según demanda
- **Costo eficiente:** Pago por uso (CPU-seconds + memory)
- **Despliegue rápido:** Rollout en segundos
- **Integración CI/CD:** Deploy automático desde GitHub

Flujo de Request:

Internet → Load Balancer → Cloud Armor → Cloud Run Instance → DB/Services

2. Cloud SQL (PostgreSQL)

Descripción:

Base de datos PostgreSQL managed con alta disponibilidad.

Configuración:

```

instance: crtlpyme-db-prod
databaseVersion: POSTGRES_15
tier: db-g1-small # 1 vCPU, 1.7 GB RAM

settings:
  availabilityType: REGIONAL # Multi-AZ
  backupConfiguration:
    enabled: true
    startTime: "02:00"
    pointInTimeRecoveryEnabled: true
    transactionLogRetentionDays: 7

  ipConfiguration:
    ipv4Enabled: false
    privateNetwork: projects/crtlpyme/global/networks/default

  databaseFlags:
    - name: max_connections
      value: "100"
    - name: shared_buffers
      value: "256MB"
    - name: work_mem
      value: "4MB"

```

Seguridad:

- **Private IP:** No expuesto a internet
- **Cloud SQL Proxy:** Conexión segura desde Cloud Run
- **Backups automáticos:** Diarios con retention de 30 días
- **Point-in-time recovery:** Restauración a cualquier momento en los últimos 7 días

Connection Pooling:

```

// Prisma configurado con connection pooling
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
  directUrl = env("DIRECT_DATABASE_URL")
}

// Connection pool settings
const prisma = new PrismaClient({
  datasources: {
    db: {
      url: process.env.DATABASE_URL,
    },
  },
  log: ['error', 'warn'],
});

```

3. Cloud Storage

Descripción:

Almacenamiento de objetos para archivos estáticos y uploads de usuarios.

Buckets:

```
# Bucket para assets estáticos (CSS, JS, imágenes)
static-assets:
  location: US
  storageClass: STANDARD
  publicAccessPrevention: enforced
  uniformBucketLevelAccess: true
  cors:
    - origin: ["https://crtlpyme.com"]
      method: ["GET"]
      maxAgeSeconds: 3600

# Bucket para uploads de usuarios
user-uploads:
  location: US
  storageClass: STANDARD
  publicAccessPrevention: enforced
  lifecycleRules:
    - action: Delete
      condition:
        age: 365 # Eliminar archivos después de 1 año
```

Integración con CDN:

- Assets estáticos servidos vía Cloud CDN
- Cache global con edge locations
- Invalidación automática en deployments

4. Cloud Load Balancer + Cloud CDN

Descripción:

Balanceador de carga HTTPS global con CDN integrado.

Configuración:

```
loadBalancer:
  type: HTTPS
  ipAddress: 34.120.xxx.xxx
  certificateType: GOOGLE_MANAGED
  domains:
    - crtlpyme.com
    - www.crtlpyme.com

cdn:
  enabled: true
  cacheMode: CACHE_ALL_STATIC
  defaultTtl: 3600
  maxTtl: 86400
  clientTtl: 3600

securityPolicy:
  cloudArmor: crtlpyme-security-policy
```

Cloud Armor (DDoS Protection):

```

securityPolicy:
  name: crtlyme-security-policy
  rules:
    - priority: 1000
      action: allow
      match:
        versionedExpr: SRC_IPS_V1
        config:
          srcIpRanges: ["*"]
      rateLimitOptions:
        conformAction: allow
        exceedAction: deny-403
        enforceOnKey: IP
        rateLimitThreshold:
          count: 100
          intervalSec: 60

    - priority: 2000
      action: deny-403
      match:
        expr:
          expression: "origin.region_code == 'CN' || origin.region_code == 'RU'"
      description: "Block traffic from high-risk countries"

```

5. Identity Platform / NextAuth.js

Descripción:

Autenticación de usuarios con NextAuth.js, opcionalmente respaldado por GCP Identity Platform.

NextAuth Configuration:


```
// lib/auth/nextauth.ts
export const authOptions: NextAuthOptions = {
  adapter: PrismaAdapter(prisma),
  providers: [
    CredentialsProvider({
      name: 'Credentials',
      credentials: {
        email: { label: "Email", type: "email" },
        password: { label: "Password", type: "password" }
      },
      async authorize(credentials) {
        // Validar credenciales contra DB
        const user = await prisma.user.findUnique({
          where: { email: credentials.email }
        });

        if (user && await bcrypt.compare(credentials.password, user.password)) {
          return {
            id: user.id,
            email: user.email,
            role: user.role,
            tenantId: user.tenantId
          };
        }
        return null;
      }
    })
  ],
  session: {
    strategy: 'jwt',
    maxAge: 30 * 24 * 60 * 60, // 30 días
  },
  callbacks: {
    async jwt({ token, user }) {
      if (user) {
        token.role = user.role;
        token.tenantId = user.tenantId;
      }
      return token;
    },
    async session({ session, token }) {
      session.user.role = token.role;
      session.user.tenantId = token.tenantId;
      return session;
    }
  }
};
```

6. Secret Manager

Descripción:

Gestión segura de secretos y credenciales.

Secrets Almacenados:

```

secrets:
- name: database-url
  value: "postgresql://user:pass@private-ip:5432/crtlpyme"

- name: nextauth-secret
  value: "random-generated-secret"

- name: transbank-commerce-code
  value: "59705555532"

- name: transbank-api-key
  value: "579B532A7440BB0C9079DED94D31EA1615BACEB56610332264630D42D0A36B1C"

- name: sendgrid-api-key
  value: "SG.xxxxxxxxxxxxxxx"

permissions:
- serviceAccount: cloud-run-service-account@project.iam
  roles:
    - roles/secretmanager.secretAccessor

```

Acceso desde Cloud Run:

```

// Carga automática desde variables de entorno
const DATABASE_URL = process.env.DATABASE_URL; // Secret Manager → Env Var

```

7. Cloud Monitoring & Logging

Descripción:

Monitoreo, logs y trazabilidad de requests.

Métricas Monitoreadas:

- Request count y latencia (p50, p95, p99)
- Error rate (4xx, 5xx)
- CPU y memory utilization
- Database connection pool usage
- Active Cloud Run instances

Alertas Configuradas:

```

alerts:
- name: high-error-rate
  condition: error_rate > 5%
  duration: 5m
  notification: email, slack

- name: high-latency
  condition: p95_latency > 2s
  duration: 5m
  notification: email

- name: database-connections
  condition: connection_pool_usage > 80%
  duration: 3m
  notification: pagerduty

```

Log Aggregation:

```
// Structured logging para fácil búsqueda
import { logger } from '@lib/logger';

logger.info('Sale created', {
  saleId: sale.id,
  tenantId: tenant.id,
  userId: user.id,
  amount: sale.total,
  items: sale.items.length
});
```

6.4 Estrategia de Escalabilidad

Escalado Horizontal (Cloud Run)

Configuración Auto-scaling:

```
scaling:
  minInstances: 0      # Escala a 0 en horarios de baja demanda
  maxInstances: 100    # Límite superior
  targetConcurrency: 80 # Requests por instancia antes de escalar

  scaleDownDelay: 5m   # Tiempo antes de eliminar instancia ociosa
```

Escenarios:

Escenario 1: Horario Normal (10 AM - 8 PM)

- Tráfico: ~200 req/min
- Instancias activas: 2-3
- Latencia: <200ms

Escenario 2: Peak (12 PM - 2 PM)

- Tráfico: ~800 req/min
- Instancias activas: 8-10
- Latencia: <300ms

Escenario 3: Madrugada (2 AM - 6 AM)

- Tráfico: ~5 req/min
- Instancias activas: 0
- Latencia: <500ms (cold start)

Escalado Vertical (Database)

Upgrade Path:

```
db-g1-small (1 vCPU, 1.7GB)
  ↓
db-custom-2-7680 (2 vCPU, 7.5GB)
  ↓
db-custom-4-15360 (4 vCPU, 15GB)
  ↓
db-custom-8-30720 (8 vCPU, 30GB)
```

Trigger para upgrade:

- CPU usage > 80% sostenido por 10 minutos

- Connection pool saturation (>90% uso)
- Query latency degradation

Read Replicas

Para operaciones de solo lectura:

```
readReplicas:
  - name: crtlpyme-db-read-replica-1
    region: us-east1
    tier: db-g1-small

  - name: crtlpyme-db-read-replica-2
    region: us-west1 # Diferente región
    tier: db-g1-small
```

Load Balancing de Queries:

```
// Prisma con replicas
const prismaRead = new PrismaClient({
  datasources: {
    db: { url: process.env.DATABASE_READ_REPLICA_URL }
  }
});

// Queries de solo lectura a replicas
const products = await prismaRead.product.findMany({
  where: { tenantId }
});

// Escrituras al primary
const sale = await prisma.sale.create({ data: {...} });
```

CDN y Caching

Niveles de Cache:

1. **Browser Cache:** Assets estáticos (JS, CSS, images)
 - Cache-Control: max-age=31536000, immutable
2. **CDN Edge Cache:** Cloud CDN
 - Cache-Control: public, max-age=3600
 - Invalidación automática en deploy
3. **Application Cache:** Redis/Memcached (futuro)
 - Product catalog cache
 - User sessions
 - Rate limiting counters

Cache Strategy:

```
// Next.js cache configuration
export const revalidate = 3600; // ISR cada 1 hora

// API route caching
export async function GET() {
  return NextResponse.json(data, {
    headers: {
      'Cache-Control': 'public, s-maxage=3600, stale-while-revalidate=86400'
    }
  });
}
```

Proyección de Costos (Estimado)

Tier Gratuito (Development):

- Cloud Run: 2 millones de requests/mes gratis
- Cloud SQL: db-f1-micro gratis por 720 horas/mes
- Storage: 5GB gratis
- **Costo total:** \$0/mes

Tier Producción (100 tenants, 10K req/día):

- Cloud Run: ~\$20/mes
- Cloud SQL (db-g1-small): ~\$25/mes
- Storage (50GB): ~\$1/mes
- Load Balancer: ~\$18/mes
- **Costo total:** ~\$64/mes

Tier Escalado (1000 tenants, 100K req/día):

- Cloud Run: ~\$150/mes
- Cloud SQL (db-custom-2-7680): ~\$90/mes
- Storage (200GB): ~\$5/mes
- Load Balancer + CDN: ~\$50/mes
- **Costo total:** ~\$295/mes

Vista de Escenarios (Casos de Uso)

7.1 Descripción General

La Vista de Escenarios (también llamada Vista de Casos de Uso o “+1”) es el elemento **integrador del Modelo 4+1**. Esta vista:

- Identifica los **casos de uso más importantes** del sistema
- Valida que las otras cuatro vistas satisfacen los requisitos
- Sirve como **punto de partida** para descubrir elementos arquitectónicos
- Facilita la **comunicación con stakeholders no técnicos**

Los escenarios representan las **secuencias de interacciones** entre actores externos y el sistema, describiendo el comportamiento de extremo a extremo.

7.2 Actores del Sistema

```
graph LR
    subgraph "Actores Principales"
        Admin[ Administrador<br/>ADMIN]
        Cashier[ Cajero<br/>CASHIER]
        Manager[ Supervisor<br/>MANAGER]
    end

    subgraph "Actores Secundarios"
        Customer[ Cliente]
        Provider[ Proveedor SaaS]
        System[ Sistema Automatizado]
    end

    subgraph "Sistemas Externos"
        Transbank[ Transbank]
        Email[ Email Service]
    end

    Admin -->|Gestiona| Manager
    Admin -->|Gestiona| Cashier
    Manager -->|Supervisa| Cashier

    Cashier -->|Atiende| Customer

    Provider -->|Administra| Admin

    System -->|Notifica| Email
    System -->|Procesa pagos| Transbank

    style Admin fill:#e8f5e9
    style Cashier fill:#e1f5ff
    style Manager fill:#fff4e1
    style Provider fill:#f3e5f5
    style System fill:#ffebee
```

Descripción de Actores:

1. Administrador (ADMIN)

- **Rol:** Dueño o encargado del negocio
- **Responsabilidades:**
 - Configurar el negocio y usuarios
 - Gestionar inventario completo
 - Ver reportes y analytics
 - Configurar punto de equilibrio
 - Administrar clientes y suscripciones
- **Permisos:** Acceso completo al tenant

2. Cajero (CASHIER)

- **Rol:** Empleado que atiende en caja
- **Responsabilidades:**
 - Registrar ventas en el POS
 - Consultar información de productos
 - Ver historial de ventas propias
- **Permisos:** Solo módulo de ventas (lectura de inventario)

3. Supervisor (MANAGER)

- **Rol:** Encargado de turno
- **Responsabilidades:**
 - Supervisar ventas del día
 - Ver reportes en tiempo real
 - Gestionar inventario (lectura)
 - Autorizar ajustes menores
- **Permisos:** Lectura completa, escritura limitada

4. Cliente

- **Rol:** Usuario externo que contrata el servicio SaaS
- **Responsabilidades:**
 - Registrarse en la plataforma
 - Seleccionar plan de suscripción
 - Realizar pagos
- **Interacción:** Landing page y onboarding

5. Proveedor SaaS

- **Rol:** Operador de la plataforma CRTLPyme
- **Responsabilidades:**
 - Gestionar suscripciones de tenants
 - Aprobar solicitudes de cajas extra
 - Monitorear salud del sistema
 - Soporte técnico
- **Permisos:** Acceso administrativo global

6. Sistema Automatizado

- **Rol:** Procesos programados (cron jobs)
- **Responsabilidades:**
 - Facturación diaria de suscripciones
 - Envío de alertas de stock
 - Generación de reportes automáticos
 - Backup de base de datos

7.3 Diagrama de Casos de Uso


```

graph TB
  subgraph "Sistema CRTLPyme"
    subgraph "Gestión de Ventas"
      UC1[UC-1: Registrar Venta]
      UC2[UC-2: Consultar Producto<br/>por Código de Barras]
      UC3[UC-3: Ver Historial<br/>de Ventas]
    end

    subgraph "Gestión de Inventario"
      UC4[UC-4: Crear/Editar Producto]
      UC5[UC-5: Ajustar Stock]
      UC6[UC-6: Importar Catálogo Masivo]
      UC7[UC-7: Ver Alertas de Stock Bajo]
    end

    subgraph "Gestión de Clientes"
      UC8[UC-8: Registrar Cliente]
      UC9[UC-9: Crear Suscripción]
      UC10[UC-10: Procesar Pago<br/>de Suscripción]
    end

    subgraph "Análisis Financiero"
      UC11[UC-11: Registrar<br/>Gasto Fijo]
      UC12[UC-12: Registrar<br/>Gasto Variable]
      UC13[UC-13: Ver Dashboard<br/>de Punto de Equilibrio]
      UC14[UC-14: Consultar Histórico<br/>de Punto de Equilibrio]
      UC15[UC-15: Generar Reporte<br/>de Ventas]
    end

    subgraph "Gestión de Usuarios"
      UC16[UC-16: Invitar Usuario]
      UC17[UC-17: Login al Sistema]
      UC18[UC-18: Cambiar Contraseña]
    end

    subgraph "Onboarding SaaS"
      UC19[UC-19: Registrar Nuevo<br/>Negocio (Tenant)]
      UC20[UC-20: Seleccionar Plan<br/>de Suscripción]
      UC21[UC-21: Inscribir Tarjeta<br/>de Pago]
    end

    subgraph "Administración Proveedor"
      UC22[UC-22: Aprobar Solicitud<br/>de Cajas Extra]
      UC23[UC-23: Suspende/Reactivar<br/>Tenant]
      UC24[UC-24: Ver Dashboard<br/>del Proveedor]
    end

    subgraph "Procesos Automáticos"
      UC25[UC-25: Cálculo Diario<br/>Punto de Equilibrio]
      UC26[UC-26: Facturar<br/>Suscripciones]
      UC27[UC-27: Enviar Alertas<br/>de Stock]
      UC28[UC-28: Notificar Pago<br/>Fallido]
      UC29[UC-29: Enviar Alertas<br/>de Punto de Equilibrio]
    end
  end

  Cashier[👤 Cajero] --> UC1
  Cashier --> UC2
  Cashier --> UC3

  Admin[👤 Administrador] --> UC1
  Admin --> UC2

```

```

Admin --> UC3
Admin --> UC4
Admin --> UC5
Admin --> UC6
Admin --> UC7
Admin --> UC8
Admin --> UC9
Admin --> UC11
Admin --> UC12
Admin --> UC13
Admin --> UC14
Admin --> UC15
Admin --> UC16
Admin --> UC18

Manager[ Supervisor] --> UC3
Manager --> UC7
Manager --> UC13
Manager --> UC14
Manager --> UC15

Customer[ Cliente] --> UC19
Customer --> UC20
Customer --> UC21

Provider[ Proveedor] --> UC22
Provider --> UC23
Provider --> UC24

System[ Sistema] --> UC25
System --> UC26
System --> UC27
System --> UC28
System --> UC29

UC1 -->|actualiza| UC7
UC1 -->|genera| UC3
UC1 -->|afecta| UC13
UC4 -->|afecta| UC7
UC5 -->|afecta| UC7
UC9 -->|requiere| UC10
UC11 -->|afecta| UC13
UC12 -->|afecta| UC13
UC19 -->|incluye| UC20
UC20 -->|requiere| UC21
UC25 -->|puede generar| UC29
UC26 -->|puede generar| UC28

style UC1 fill:#e8f5e9
style UC4 fill:#e1f5ff
style UC11 fill:#fff4e1
style UC12 fill:#fff4e1
style UC13 fill:#fff4e1
style UC19 fill:#f3e5f5
style UC25 fill:#ffebee
style UC26 fill:#ffebee

```

7.4 Especificación de Casos de Uso Principales

UC-1: Registrar Venta (POS)

Campo	Descripción
ID	UC-1
Nombre	Registrar Venta en Punto de Venta
Actor Principal	Cajero (CASHIER)
Actores Secundarios	Cliente
Precondiciones	- Usuario autenticado con rol CASHIER - Al menos un producto en inventario
Postcondiciones	- Venta registrada en BD - Stock actualizado - Comprobante generado
Trigger	Cajero inicia nueva venta

Flujo Principal:

1. Cajero hace clic en “Nueva Venta”
2. Sistema crea carrito vacío en memoria
3. Cajero escanea código de barras del producto
4. Sistema busca producto por código de barras
5. Sistema muestra información del producto (nombre, precio, stock)
6. Cajero confirma cantidad (default: 1)
7. Sistema agrega producto al carrito
8. **Repetir pasos 3-7** para cada producto
9. Cajero revisa total y selecciona método de pago (efectivo/débito/crédito)
10. Cajero confirma venta
11. Sistema valida disponibilidad de stock para todos los ítems
12. Sistema crea registro de venta en transacción
13. Sistema actualiza stock de cada producto (decremento)
14. Sistema registra ítems de venta
15. Sistema confirma transacción (COMMIT)
16. Sistema muestra comprobante en pantalla
17. Cajero imprime comprobante (opcional)

Flujos Alternativos:

A1: Producto no encontrado (paso 4)

- 4a. Sistema muestra mensaje “Producto no encontrado”
- 4b. Cajero puede buscar manualmente por nombre o SKU
- 4c. Si encuentra, continuar en paso 5
- 4d. Si no encuentra, volver a paso 3

A2: Stock insuficiente (paso 11)

- 11a. Sistema detecta stock insuficiente para uno o más productos

- 11b. Sistema muestra alerta especificando productos problemáticos
- 11c. Cajero ajusta cantidades o elimina productos
- 11d. Volver a paso 11

A3: Error en transacción (paso 15)

- 15a. Sistema detecta error al guardar (DB timeout, constraint violation)
- 15b. Sistema ejecuta ROLLBACK
- 15c. Sistema muestra mensaje de error al cajero
- 15d. Cajero puede reintentar venta

Excepciones:

- **E1:** Usuario no autenticado → Redirigir a login
- **E2:** Usuario sin permisos → Mostrar “Acceso denegado”
- **E3:** Carrito vacío al confirmar → Mostrar “Agregar al menos un producto”

Requisitos No Funcionales:

- **RNF-1:** Tiempo de respuesta < 200ms para búsqueda de productos
- **RNF-2:** Transacción de venta completa < 2s
- **RNF-3:** Sistema debe manejar concurrencia (múltiples cajas simultáneas)

Variaciones Tecnológicas:

- Escáner USB de código de barras
- Entrada manual de código
- Búsqueda por nombre/SKU

UC-4: Crear/Editar Producto

Campo	Descripción
ID	UC-4
Nombre	Crear o Editar Producto en Inventario
Actor Principal	Administrador (ADMIN)
Actores Secundarios	Ninguno
Precondiciones	- Usuario autenticado con rol ADMIN
Postcondiciones	- Producto creado/actualizado en BD - Cambios reflejados inmediatamente en POS
Trigger	Admin accede a “Inventario” → “Nuevo Producto” o edita existente

Flujo Principal (Crear):

1. Admin hace clic en “Nuevo Producto”
2. Sistema muestra formulario vacío
3. Admin ingresa datos:
 - SKU (único)

- Código de barras (único)
- Nombre
- Categoría
- Costo (precio de compra)
- Precio de venta
- Stock inicial
- Stock mínimo (umbral de alerta)
- 4. Sistema valida datos (Zod schema)
- 5. Sistema verifica unicidad de SKU y código de barras
- 6. Sistema calcula margen automáticamente: $(\text{precioVenta} - \text{costo}) / \text{precioVenta} * 100$
- 7. Sistema guarda producto en BD
- 8. Sistema muestra mensaje de confirmación
- 9. Sistema redirige a lista de productos

Flujo Principal (Editar):

1. Admin busca producto en lista
2. Admin hace clic en "Editar"
3. Sistema carga datos actuales en formulario
4. Admin modifica campos deseados
5. Sistema valida datos
6. Sistema verifica unicidad si cambió SKU/código de barras
7. Sistema actualiza registro en BD
8. Sistema registra cambio en audit_log
9. Sistema muestra mensaje de confirmación

Flujos Alternativos:

A1: SKU o código de barras duplicado (paso 5)

- 5a. Sistema detecta duplicado
- 5b. Sistema muestra error: "SKU/Código ya existe en el sistema"
- 5c. Volver a paso 3

A2: Precio de venta menor que costo (paso 6)

- 6a. Sistema detecta margen negativo
- 6b. Sistema muestra advertencia: "Precio de venta menor que costo"
- 6c. Admin puede confirmar o corregir

Validaciones:

```
const productSchema = z.object({
  sku: z.string().min(3).max(50),
  barcode: z.string().min(8).max(20),
  name: z.string().min(3).max(200),
  category: z.string().optional(),
  cost: z.number().positive(),
  price: z.number().positive(),
  stock: z.number().int().nonnegative(),
  minStock: z.number().int().nonnegative()
});
```

UC-11: Configurar Punto de Equilibrio

Campo	Descripción
ID	UC-11
Nombre	Configurar Cálculo de Punto de Equilibrio
Actor Principal	Administrador (ADMIN)
Actores Secundarios	Sistema (cálculo automático)
Precondiciones	- Usuario autenticado con rol ADMIN - Tenant creado
Postcondiciones	- Configuración de breakeven guardada - Dashboard muestra progreso hacia punto de equilibrio
Trigger	Admin accede a “Configuración” → “Punto de Equilibrio”

Flujo Principal:

- Admin accede a configuración de punto de equilibrio
- Sistema muestra formulario con campos:
 - **Costos Fijos Mensuales** (arriendo, servicios, salarios)
 - **Margen Promedio (%)** - puede ser calculado automáticamente
- Admin ingresa costos fijos mensuales (ej: \$500,000 CLP)
- Admin puede:
 - Ingresar margen promedio manualmente (ej: 30%)
 - O solicitar cálculo automático basado en productos existentes
- Si selecciona cálculo automático:
 - Sistema obtiene todos los productos del tenant
 - Sistema calcula margen por producto: $(\text{precio} - \text{costo}) / \text{precio}$
 - Sistema calcula promedio ponderado por stock o por cantidad vendida
- Sistema calcula punto de equilibrio:

$$\text{Punto de Equilibrio (ventas mensuales)} = \text{Costos Fijos} / \text{Margen Promedio}$$
- Sistema muestra resultados:
 - Monto de ventas necesario por mes
 - Ventas necesarias por día (asumiendo 30 días)
 - Progreso actual (ventas acumuladas del mes)
- Admin confirma y guarda configuración
- Sistema actualiza `breakeven_configs` en BD
- Sistema actualiza dashboard principal

Ejemplo de Cálculo:

Costos Fijos: \$600,000 CLP/mes
Margen Promedio: 35%

Punto de Equilibrio = $\$600,000 / 0.35 = \$1,714,286$ CLP/mes

Diario = $\$1,714,286 / 30 = \$57,143$ CLP/día

Si ventas acumuladas del mes = \$400,000
Progreso = $(\$400,000 / \$1,714,286) * 100 = 23.3\%$

Dashboard Visualization:

```
graph LR
  A[Ventas Actuales<br/>$400,000] -->|23.3%| B[Punto de Equilibrio<br/>$1,714,286]

  style A fill:#ffebee
  style B fill:#e8f5e9
```

UC-17: Registrar Nuevo Negocio (Onboarding)

Campo	Descripción
ID	UC-17
Nombre	Registrar Nuevo Negocio en la Plataforma SaaS
Actor Principal	Cliente (Prospecto)
Actores Secundarios	Transbank (pago), Email Service (notificación)
Precondiciones	- Ninguna
Postcondiciones	- Tenant creado - Usuario admin creado - Suscripción activa - Tarjeta inscrita en Transbank - Email de bienvenida enviado
Trigger	Cliente hace clic en “Crear Cuenta” en landing page

Flujo Principal:

Paso 1: Selección de Plan

1. Cliente accede a página de planes

2. Sistema muestra opciones:

- Plan Básico: \$15,000/mes (2 cajas incluidas)

- Plan Pro: \$30,000/mes (5 cajas incluidas)

- Opción de cajas adicionales: \$5,000/mes por caja

3. Cliente selecciona plan
4. Cliente indica si necesita cajas extra
5. Sistema calcula precio total mensual
6. Cliente hace clic en "Continuar"

Paso 2: Datos del Negocio

7. Sistema muestra formulario:
 - Nombre del negocio
 - RUT/Tax ID
 - Tipo de negocio (dropdown)
 - Dirección
 - Teléfono
8. Cliente completa formulario
9. Sistema valida datos
10. Cliente hace clic en "Continuar"

Paso 3: Datos del Administrador

11. Sistema solicita:
 - Nombre completo
 - Email
 - Contraseña (con requisitos de seguridad)
12. Cliente completa datos
13. Sistema valida:
 - Email único (no existe en sistema)
 - Contraseña cumple requisitos (min 8 chars, uppercase, números)
14. Cliente hace clic en "Continuar"

Paso 4: Configuración de Punto de Equilibrio

15. Sistema explica concepto de punto de equilibrio
16. Sistema solicita:
 - Costos fijos mensuales
 - Margen promedio estimado (%)
17. Cliente ingresa valores
18. Sistema calcula y muestra punto de equilibrio estimado
19. Cliente hace clic en "Continuar"

Paso 5: Inscripción de Tarjeta (Transbank)

20. Sistema inicia flujo Oneclick de Transbank
21. Sistema genera token de inscripción
22. Sistema redirige a página de Transbank
23. Cliente ingresa datos de tarjeta en Transbank
24. Transbank valida tarjeta (sin cobro)
25. Transbank retorna a callback de CRTLPyme
26. Sistema confirma inscripción con Transbank
27. Sistema guarda token de pago

Paso 6: Creación de Tenant (Transacción Atómica)

28. Sistema inicia transacción de BD
29. Sistema crea registro en `tenants`
30. Sistema crea usuario admin en `users`
31. Sistema crea configuración en `breakeven_configs`
32. Sistema crea suscripción en `subscriptions` (estado: ACTIVE)

33. Sistema establece `next_billing_date` (30 días desde hoy)

34. Sistema confirma transacción (COMMIT)

Paso 7: Confirmación y Acceso

35. Sistema envía email de bienvenida con:

- URL de acceso
- Credenciales temporales
- Guía de inicio rápido

36. Sistema muestra página de confirmación

37. Sistema ofrece “Ir al Dashboard” o “Ver Tutorial”

Flujos Alternativos:

A1: Email ya existe (paso 13)

- 13a. Sistema detecta email duplicado
- 13b. Sistema muestra: “Email ya registrado. ¿Olvidaste tu contraseña?”
- 13c. Ofrecer link de recuperación de contraseña

A2: Error en inscripción de tarjeta (paso 26)

- 26a. Transbank retorna error (tarjeta rechazada)
- 26b. Sistema muestra mensaje explicativo
- 26c. Sistema ofrece reintentar con otra tarjeta
- 26d. Si cliente cancela, volver a paso 20

A3: Error al crear tenant (paso 34)

- 34a. Sistema detecta error en transacción
- 34b. Sistema ejecuta ROLLBACK
- 34c. Sistema registra error en logs
- 34d. Sistema muestra mensaje: “Error al crear cuenta. Por favor intenta nuevamente”

Requisitos No Funcionales:

- **RNF-1:** Proceso completo < 5 minutos
 - **RNF-2:** Transacción atómica (todo o nada)
 - **RNF-3:** Inscripción de tarjeta en ambiente sandbox (Transbank)
 - **RNF-4:** Email enviado en < 30 segundos
-

UC-23: Facturar Suscripciones (Automatizado)

Campo	Descripción
ID	UC-23
Nombre	Facturación Automática de Suscripciones
Actor Principal	Sistema Automatizado (Cron Job)
Actores Secundarios	Transbank, Email Service, Administradores de Tenants
Precondiciones	- Suscripciones activas con <code>next_billing_date = today</code>
Postcondiciones	- Cobros procesados - Registros de pago creados - <code>next_billing_date</code> actualizado - Emails de comprobante enviados
Trigger	GitHub Actions cron job (diario a las 00:00 UTC)

Flujo Principal:**Inicialización:**

1. Job se activa a las 00:00 UTC
2. Sistema adquiere lock distribuido para evitar ejecuciones concurrentes
3. Sistema registra inicio de ejecución en logs

Procesamiento por Lotes:

4. Sistema consulta suscripciones pendientes:

```
sql
```

```
SELECT * FROM subscriptions
WHERE status = 'ACTIVE'
AND next_billing_date = CURRENT_DATE
ORDER BY tenant_id;
```

5. Sistema obtiene N suscripciones (ej: 100)
6. Para cada suscripción:

Procesamiento Individual:

7. Sistema verifica estado del tenant
8. Si tenant suspendido, saltar al siguiente
9. Sistema obtiene token de Transbank de la suscripción
10. Sistema genera ID único de transacción (idempotency key)
11. Sistema verifica que no se haya procesado anteriormente
12. Sistema calcula monto a cobrar:
 - Precio base del plan
 - + Costo de cajas extra
 - + IVA (19% en Chile)

13. Sistema llama a Transbank Oneclick Charge:

```
typescript
const response = await transbank.oneclick.authorize({
  buyOrder: generateBuyOrder(),
  tbkUser: subscription.transbank_token,
  username: subscription.tenant.name,
  amount: totalAmount
});
```

Manejo de Respuesta:

14. Si cobro exitoso (responseCode = 0):

- 14a. Sistema crea registro en `subscription_payments` :
 - `status = 'COMPLETED'`
 - `amount = totalAmount`
 - `transaction_id = response.authorizationCode`
 - `payment_date = NOW()`
- 14b. Sistema actualiza suscripción:
 - `next_billing_date = next_billing_date + interval (30 días)`
 - `last_payment_date = NOW()`
- 14c. Sistema envía email de comprobante al admin del tenant
- 14d. Marcar como procesado en tabla de idempotencia

1. Si cobro fallido (responseCode != 0):

- 15a. Sistema registra intento fallido
- 15b. Sistema incrementa contador de reintentos
- 15c. Si reintentos < 3:
 - Programar reintento para mañana
 - Enviar email de alerta al admin
- 15d. Si reintentos >= 3:
 - Actualizar suscripción: `status = 'SUSPENDED'`
 - Enviar email de suspensión al admin
 - Registrar en `audit_log`

Finalización:

- 16. Sistema procesa siguiente suscripción
- 17. Cuando termina el lote, verificar si hay más pendientes
- 18. Si hay más, volver a paso 5
- 19. Si no hay más, generar reporte de ejecución:
 - Total procesados
 - Exitosos
 - Fallidos
 - Monto total cobrado
- 20. Sistema envía reporte al equipo de operaciones
- 21. Sistema libera lock distribuido
- 22. Sistema registra fin de ejecución

Configuración GitHub Actions:

```

name: Daily Billing
on:
  schedule:
    - cron: '0 0 * * *' # Diario a medianoche UTC
  workflow_dispatch:    # Manual trigger

jobs:
  billing:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Setup Node
        uses: actions/setup-node@v3
        with:
          node-version: '18'
      - name: Run Billing
        env:
          DATABASE_URL: ${ secrets.DATABASE_URL }
          TRANSBANK_API_KEY: ${ secrets.TRANSBANK_API_KEY }
        run: npm run billing:process

```

Manejo de Errores:

- **Timeout de Transbank:** Reintentar después de 1 minuto
- **DB Connection Lost:** Reintentar con exponential backoff
- **Lock Acquisition Failed:** Abortar ejecución (otra instancia corriendo)

Decisiones Arquitectónicas

8.1 Arquitectura Serverless

Decisión: Adoptar arquitectura serverless con Cloud Run y Next.js API Routes.

Contexto:

El sistema debe ser escalable, con costos predecibles y bajos para startups. Los negocios pequeños tienen patrones de uso variables (peaks en horarios de atención, inactividad en madrugada).

Alternativas Consideradas:

Opción	Pros	Contras
Monolito en VM	Simple, control total	Requiere gestión de servidores, escalado manual, costos fijos altos
Kubernetes	Escalabilidad, orquestación avanzada	Complejidad operativa, overhead de gestión, costos
Serverless (Elegida)	Auto-scaling, costo por uso, sin gestión de servidores	Cold starts, límites de ejecución

Justificación:

- **Escalado a cero:** Costos mínimos cuando no hay tráfico

- **Escalado automático:** Maneja peaks sin intervención manual
- **Time-to-market:** Deploy rápido sin configuración de infraestructura
- **Desarrollo ágil:** CI/CD simplificado

Trade-offs Aceptados:

- Cold starts (~500ms) aceptables para aplicación de negocio
- Límite de tiempo de ejecución (60s en Cloud Run) suficiente para operaciones transaccionales
- Sin estado en memoria (usar DB/cache para session)

8.2 Multi-Tenancy

Decisión: Implementar multi-tenancy con modelo “Shared Database, Separate Schema” (discriminador `tenant_id`).

Contexto:

Plataforma SaaS que sirve a múltiples negocios independientes. Necesidad de:

- Aislamiento total de datos entre clientes
- Eficiencia de recursos (costo)
- Facilidad de deployment y mantenimiento

Alternativas Consideradas:

Modelo	Aislamiento	Costo	Complejidad	Decisión
Base de datos separada por tenant	★★★★★	💰💰💰💰💰	🔧🔧🔧🔧	✗
Schema separado por tenant	★★★★★	💰💰💰	🔧🔧🔧	✗
Tabla compartida + tenant_id	★★★	💰	🔧🔧	✓

Implementación:

```
// Middleware que inyecta tenant_id en contexto
export async function withTenant(
  req: Request,
  handler: (tenantId: string) => Promise<Response>
) {
  const session = await getSession(req);
  if (!session?.user?.tenantId) {
    return new Response('Unauthorized', { status: 401 });
  }

  return handler(session.user.tenantId);
}

// Uso en API route
export async function GET(req: Request) {
  return withTenant(req, async (tenantId) => {
    const products = await prisma.product.findMany({
      where: { tenantId } // Automático en todas las queries
    });
    return Response.json(products);
  });
}
```

Prisma Middleware para Aislamiento:

```
prisma.$use(async (params, next) => {
  // Auto-inyectar tenant_id en queries
  if (params.model && params.args?.where) {
    if (!params.args.where.tenantId) {
      throw new Error('tenantId requerido en query');
    }
  }
  return next(params);
});
```

Ventajas:

- **Costo-efectivo:** Una sola base de datos para todos los tenants
- **Mantenimiento simple:** Schema migrations aplicadas una vez
- **Backup unificado:** Un backup cubre todos los tenants
- **Performance:** Índices compartidos, query optimizer eficiente

Seguridad:

- **Row-Level Security (RLS) a nivel aplicación:** Middleware obligatorio
- **Índices compuestos:** `CREATE INDEX ON products(tenant_id, sku);`
- **Foreign keys con tenant_id:** Evita referencias cruzadas accidentales

8.3 Seguridad

8.3.1 Autenticación y Autorización

Decisión: NextAuth.js con JWT y RBAC (Role-Based Access Control).

Flujo de Autenticación:

```
sequenceDiagram
    actor User
    participant App
    participant NextAuth
    participant DB

    User->>App: POST /api/auth/signin<br/>{email, password}
    App->>NextAuth: Credentials Provider
    NextAuth->>DB: SELECT user WHERE email
    DB-->>NextAuth: User data
    NextAuth->>NextAuth: bcrypt.compare(password, hash)
    alt Password válido
        NextAuth->>NextAuth: Generar JWT
        Note over NextAuth: Claims: {userId, tenantId, role}
        NextAuth-->>App: Set-Cookie: next-auth.session-token
        App-->>User: Redirect /dashboard
    else Password inválido
        NextAuth-->>App: Error
        App-->>User: "Credenciales inválidas"
    end
```

JWT Claims:

```
interface JWT {
  sub: string;           // user.id
  tenantId: string;
  role: 'ADMIN' | 'CASHIER' | 'MANAGER';
  iat: number;
  exp: number;
}
```

RBAC Matrix:

Recurso	ADMIN	MANAGER	CASHIER
Ver productos	✓	✓	✓ (solo consulta)
Crear/editar productos	✓	✗	✗
Eliminar productos	✓	✗	✗
Registrar venta	✓	✓	✓
Ver historial ventas	✓ (todas)	✓ (todas)	✓ (propias)
Ver reportes	✓	✓	✗
Gestionar usuarios	✓	✗	✗
Configurar breakeven	✓	✗	✗

Implementación de Permisos:

```
// lib/auth/permissions.ts
export function can(
  userRole: UserRole,
  action: string,
  resource: string
): boolean {
  const permissions = {
    ADMIN: ['*'], // Full access
    MANAGER: ['read:*', 'create:sale', 'read:reports'],
    CASHIER: ['read:products', 'create:sale', 'read:own_sales']
  };

  const userPerms = permissions[userRole];
  return userPerms.includes('*') ||
    userPerms.includes(`${action}:${resource}`) ||
    userPerms.includes(`${action}:*`);
}

// Uso en API Route
export async function DELETE(req: Request, { params }: { params: { id: string } }) {
  const session = await getSession(req);

  if (!can(session.user.role, 'delete', 'product')) {
    return new Response('Forbidden', { status: 403 });
  }

  await prisma.product.delete({
    where: {
      id: params.id,
      tenantId: session.user.tenantId // Tenant isolation
    }
  });

  return new Response(null, { status: 204 });
}
```

8.3.2 Seguridad en Tránsito y Reposo

En Tránsito:

- **HTTPS obligatorio:** TLS 1.3 en Cloud Load Balancer
- **HSTS:** Strict-Transport-Security header
- **CSP:** Content-Security-Policy para XSS prevention

En Reposo:

- **Cloud SQL:** Encriptación automática (AES-256)
- **Passwords:** bcrypt con salt rounds = 12
- **Secrets:** Secret Manager con IAM controls
- **Backups:** Encriptados automáticamente

8.3.3 Mitigación de Vulnerabilidades

SQL Injection:

- ☒ **Prisma ORM:** Prepared statements automáticos
- ☒ **Validación:** Zod schemas en inputs

XSS (Cross-Site Scripting):

- ☒ **React:** Auto-escaping por default
- ☒ **CSP Headers:** Content-Security-Policy
- ☒ **Sanitización:** DOMPurify para user-generated content

CSRF (Cross-Site Request Forgery):

- ☒ **NextAuth:** CSRF tokens automáticos
- ☒ **SameSite Cookies:** Strict mode

Brute Force:

- ☒ **Rate Limiting:** Cloud Armor + Application-level

```
import { Ratelimit } from '@upstash/ratelimit';

const ratelimit = new Ratelimit({
  limiter: Ratelimit.slidingWindow(5, '1m'), // 5 req/min
});

export async function POST(req: Request) {
  const ip = req.headers.get('x-forwarded-for') || 'unknown';
  const { success } = await ratelimit.limit(ip);

  if (!success) {
    return new Response('Too Many Requests', { status: 429 });
  }
  // ...
}
```

8.4 Escalabilidad y Rendimiento

8.4.1 Estrategias de Caching

Niveles de Cache:

```
graph TB
  Browser[Browser Cache<br/>Assets estáticos]
  CDN[Cloud CDN<br/>Global edge cache]
  App[Application Cache<br/>Redis/Memcached]
  DB[(Database<br/>Query cache)]

  User --> Browser
  Browser --> CDN
  CDN --> App
  App --> DB

  style Browser fill:#e1f5ff
  style CDN fill:#fff4e1
  style App fill:#e8f5e9
  style DB fill:#ffebee
```

1. Browser Cache:

```
// next.config.js
module.exports = {
  async headers() {
    return [
      {
        source: '/static/:path*',
        headers: [
          {
            key: 'Cache-Control',
            value: 'public, max-age=31536000, immutable'
          }
        ]
      }
    ]
  }
};
```

2. Cloud CDN:

- Cache-Control headers en responses
- Invalidación automática en deploy
- Edge locations para latencia baja

3. Application Cache (Futuro):

```
// Redis cache para catálogo de productos
import { Redis } from '@upstash/redis';

const redis = new Redis({ url: process.env.REDIS_URL });

export async function getProducts(tenantId: string) {
  const cacheKey = `products:${tenantId}`;

  // Try cache first
  const cached = await redis.get(cacheKey);
  if (cached) return JSON.parse(cached);

  // Cache miss - fetch from DB
  const products = await prisma.product.findMany({
    where: { tenantId }
  });

  // Store in cache (1 hour TTL)
  await redis.set(cacheKey, JSON.stringify(products), { ex: 3600 });

  return products;
}
```

8.4.2 Database Optimization

Índices Estratégicos:

```
-- Índice compuesto para multi-tenancy
CREATE INDEX idx_products_tenant_sku ON products(tenant_id, sku);
CREATE INDEX idx_sales_tenant_date ON sales(tenant_id, date DESC);

-- Índice para búsqueda por código de barras
CREATE INDEX idx_products_barcode ON products(barcode) WHERE barcode IS NOT NULL;

-- Índice para alertas de stock bajo
CREATE INDEX idx_products_low_stock ON products(tenant_id, stock) WHERE stock < min_stock;
```

Connection Pooling:

```
// Prisma connection pool
const prisma = new PrismaClient({
  datasources: {
    db: {
      url: process.env.DATABASE_POOLING_URL
    }
  }
});

// Pool settings en Cloud SQL
// max_connections: 100
// Connections por Cloud Run instance: 5-10
// Max instances: 100
// Total connections: ~1000 (dentro del límite)
```

Query Optimization:

```
// ❌ N+1 query problem
const sales = await prisma.sale.findMany();
for (const sale of sales) {
  const items = await prisma.saleItem.findMany({
    where: { saleId: sale.id }
  });
}

// ✅ Eager loading con include
const sales = await prisma.sale.findMany({
  include: {
    items: {
      include: {
        product: true
      }
    },
    user: {
      select: { name: true }
    }
  }
});
```

8.4.3 Proyección de Capacidad

Escenario Base:

- 100 tenants activos
- 10 ventas/día por tenant = 1,000 ventas/día

- 50 productos promedio por tenant
- Picos de 5x en horarios de almuerzo

Estimaciones:

Métrica	Valor	Límite
Requests/día	~50,000	Cloud Run: 2M free/mes
DB Storage	~5 GB	Cloud SQL: 100GB disponible
Bandwidth	~20 GB/mes	Cloud CDN: 50GB incluido
DB Connections	~50 promedio	Límite: 100

Crecimiento a 1,000 tenants:

- DB Storage: ~50 GB (bien dentro del límite)
- Upgrade a `db-custom-2-7680` recomendado
- Considerar read replicas para reportes

Conclusiones

El **Modelo 4+1 de Vistas Arquitectónicas** aplicado a **CRTLPyme** proporciona una visión integral y multidimensional del sistema, permitiendo:

Logros Arquitectónicos

1. Separación de Concerns:

- Cada vista aborda preocupaciones específicas de diferentes stakeholders
- Vista Lógica: Funcionalidad y estructura conceptual
- Vista de Procesos: Comportamiento dinámico y concurrencia
- Vista de Desarrollo: Organización del código fuente
- Vista Física: Despliegue e infraestructura
- Vista de Escenarios: Validación desde requisitos de usuario

2. Arquitectura Cloud-Native:

- Aprovechamiento de servicios managed de GCP
- Escalabilidad automática con costos optimizados
- Alta disponibilidad sin complejidad operativa
- Despliegue continuo con CI/CD

3. Multi-Tenancy Eficiente:

- Aislamiento de datos por cliente
- Recursos compartidos para eficiencia de costos
- Modelo escalable a miles de tenants

4. Seguridad Integral:

- Autenticación robusta con NextAuth.js
- Autorización granular (RBAC)
- Encriptación en tránsito y reposo
- Mitigación de vulnerabilidades comunes (OWASP Top 10)

5. Escalabilidad y Rendimiento:

- Estrategia de caching multinivel
- Optimización de consultas a base de datos
- Connection pooling eficiente
- CDN global para baja latencia

Competencias Demostradas

Como **proyecto Capstone de Ingeniería en Informática**, este documento demuestra:

-  Diseño de arquitecturas de software complejas
-  Aplicación de patrones de diseño y mejores prácticas
-  Modelado usando notaciones estándar (UML, Mermaid)
-  Justificación técnica de decisiones arquitectónicas
-  Consideración de atributos de calidad (seguridad, escalabilidad, mantenibilidad)
-  Integración de servicios cloud modernos
-  Documentación técnica profesional

Próximos Pasos

Implementación:

1. Setup de infraestructura GCP (Cloud Run, Cloud SQL)
2. Implementación del modelo de datos (Prisma migrations)
3. Desarrollo de API Routes siguiendo las especificaciones
4. Implementación de UI con Next.js y React
5. Integración con Transbank (sandbox)
6. Testing (unitario, integración, e2e)
7. Deploy a producción con CI/CD

Módulos Opcionales:

- **Control de Punto de Equilibrio:** Implementado - Análisis financiero automático para determinar viabilidad mensual
- **Mantenedor de Clientes Frecuentes:** Diseñado - Sistema de fidelización con descuentos por tramos basados en compras acumuladas mensuales (módulo opcional activable por tenant)

Mejoras Futuras:

- Implementación de cache distribuido (Redis)
- Read replicas para reportes pesados
- Analytics avanzados con BigQuery
- App móvil nativa (Flutter/React Native)
- Módulo de análisis predictivo con ML
- Integración con más pasarelas de pago
- Sistema de notificaciones push
- Soporte multi-idioma (i18n)

Referencias Bibliográficas

1. **Kruchten, P.** (1995). "The 4+1 View Model of Architecture". IEEE Software, 12(6), 42-50.
 - Modelo original de vistas arquitectónicas

2. **Bass, L., Clements, P., & Kazman, R.** (2012). Software Architecture in Practice (3rd ed.). Addison-Wesley.
 - Fundamentos de arquitectura de software y atributos de calidad
 3. **Vernon, V.** (2013). Implementing Domain-Driven Design. Addison-Wesley.
 - Patrones DDD aplicados en CRTLPyne
 4. **Richardson, C.** (2018). Microservices Patterns. Manning Publications.
 - Patrones de arquitectura serverless y multi-tenancy
 5. **Newman, S.** (2021). Building Microservices (2nd ed.). O'Reilly Media.
 - Arquitectura de servicios y deployment en cloud
 6. **Google Cloud Documentation** (2025). Cloud Run Documentation.
<https://cloud.google.com/run/docs>
 - Documentación oficial de GCP Cloud Run
 7. **Vercel Documentation** (2025). Next.js Documentation.
<https://nextjs.org/docs>
 - Framework Next.js 15 utilizado en el proyecto
 8. **Prisma Documentation** (2025). Prisma ORM.
<https://www.prisma.io/docs>
 - ORM utilizado para acceso a datos
 9. **OWASP** (2024). OWASP Top Ten Web Application Security Risks.
<https://owasp.org/www-project-top-ten/>
 - Vulnerabilidades de seguridad consideradas
 10. **Transbank** (2024). Documentación Webpay Oneclick.
<https://www.transbankdevelopers.cl/>
 - Integración con pasarela de pagos chilena
-

Anexos

Anexo A: Glosario de Términos

Término	Definición
Tenant	Organización cliente del sistema SaaS (negocio independiente)
Multi-tenancy	Arquitectura donde una instancia del software sirve a múltiples clientes
Serverless	Modelo de ejecución donde el proveedor cloud gestiona la infraestructura
Breakeven	Punto de equilibrio financiero donde ingresos = costos
RBAC	Role-Based Access Control - Control de acceso basado en roles
JWT	JSON Web Token - Token de autenticación
ORM	Object-Relational Mapping - Mapeo objeto-relacional
ACID	Atomicity, Consistency, Isolation, Durability - Propiedades de transacciones
CDN	Content Delivery Network - Red de distribución de contenido
POS	Point of Sale - Punto de venta

Anexo B: Convenciones de Código

```
// Naming Conventions
interface UserSession { // PascalCase para interfaces
  userId: string;       // camelCase para propiedades
  tenantId: string;
}

class ProductService { // PascalCase para clases
  async findById(id: string) { // camelCase para métodos
    // ...
  }
}

const API_BASE_URL = 'https://api.example.com'; // SCREAMING_SNAKE_CASE para constantes

// File Naming
// Componentes: PascalCase.tsx - ProductCard.tsx
// Utils: kebab-case.ts - format-currency.ts
// API Routes: kebab-case/route.ts - /api/products/[id]/route.ts

// Comment Style
/**
 * Calcula el punto de equilibrio del negocio
 * @param fixedCosts - Costos fijos mensuales
 * @param averageMargin - Margen promedio (0-1)
 * @returns Monto de ventas necesario para punto de equilibrio
 */
function calculateBreakeven(fixedCosts: number, averageMargin: number): number {
  return fixedCosts / averageMargin;
}
```

Anexo C: Variables de Entorno

```
# Database
DATABASE_URL="postgresql://user:pass@localhost:5432/crtlpyme"
DIRECT_DATABASE_URL="postgresql://user:pass@localhost:5432/crtlpyme"

# Authentication
NEXTAUTH_SECRET="your-secret-key-here"
NEXTAUTH_URL="http://localhost:3000"

# Transbank (Sandbox)
TRANSBANK_COMMERCE_CODE="59705555532"
TRANSBANK_API_KEY="579B532A7440BB0C9079DED94D31EA1615BACEB56610332264630D42D0A36B1C"
TRANSBANK_ENVIRONMENT="integration"

# Email
SENDGRID_API_KEY="SG.xxxxxxxxxxxxxxxxxxxxxx"
EMAIL_FROM="noreply@crtlpyme.com"

# Cloud Storage
GCS_BUCKET_NAME="crtlpyme-storage"
GCS_PROJECT_ID="crtlpyme-prod"

# Monitoring
NEXT_PUBLIC_SENTRY_DSN="https://xxxxx@sentry.io/xxxxx"
```


Anexo D: Scripts Útiles

```
{  
  "scripts": {  
    "dev": "next dev",  
    "build": "next build",  
    "start": "next start",  
    "lint": "next lint",  
  
    "db:generate": "prisma generate",  
    "db:push": "prisma db push",  
    "db:migrate": "prisma migrate dev",  
    "db:seed": "tsx prisma/seed.ts",  
    "db:studio": "prisma studio",  
  
    "type-check": "tsc --noEmit",  
    "format": "prettier --write .",  
    "test": "vitest",  
    "test:ui": "vitest --ui",  
  
    "billing:process": "tsx scripts/process-billing.ts"  
  }  
}
```

Documento preparado por:

Equipo de Desarrollo CRTLPyme

Instituto Profesional DUOC UC

Proyecto Capstone 2025

Última actualización: Octubre 2025

Versión: 1.0